ĐẠI HỌC ĐÀ NẴNG
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA <TÊN KHOA>

———— ✦ ————

BÁO CÁO ĐỀ TÀI
MÔN KHOA HỌC DỮ LIỆU

DỰ ĐOÁN GIÁ Ô TÔ TRÊN CHỢ TỐT
BẰNG RIDGE REGRESSION VÀ RANDOM FOREST

Giảng viên hướng dẫn: <Tên giảng viên>
Sinh viên thực hiện: <Họ tên SV 1> – <MSSV>
<Họ tên SV 2> – <MSSV>
Lớp học phần: <Mã lớp>

Đà Nẵng, tháng 10 năm 2026

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI 8
1.1. Bối cảnh 8
1.2. Phát biểu bài toán 8
1.3. Mục tiêu và phạm vi 8
1.4. Quy trình thực hiện 9
1.5. Công cụ sử dụng 9
CHƯƠNG 2. THU THẬP DỮ LIỆU 10
2.1. Nguồn dữ liệu và phương pháp thu thập 10
2.2. Thu thập có trách nhiệm 10
2.3. Từ điển dữ liệu 10
2.4. Cơ chế thu thập và hệ quả cho tiền xử lý 11
CHƯƠNG 3. KHÁM PHÁ VÀ TIỀN XỬ LÝ DỮ LIỆU 12
3.1. Khám phá dữ liệu ban đầu 12
3.2. Chuẩn hóa kiểu dữ liệu và giá trị giả 14
3.3. Trích vị trí, loại cột thừa và tin trùng 15
3.4. Giá trị bất hợp lý 15
3.5. Giá trả trước / đặt cọc 16
3.6. Xử lý giá trị thiếu 16
3.7. Biến đổi log cho giá 18
3.8. Gộp nhãn hiếm theo cấp 18
3.9. Tạo đặc trưng 19
3.10. Dữ liệu mất cân bằng 19
3.11. Tổng kết tiền xử lý 20
CHƯƠNG 4. XÂY DỰNG MÔ HÌNH 21
4.1. Thiết kế thí nghiệm 21
4.1.1. Chia dữ liệu 21
4.1.2. Đặc trưng cho từng mô hình 21
4.2. Thước đo đánh giá 22

4.3. Mô hình 1: Ridge Regression 23
4.3.1. Cơ sở lý thuyết 23
4.3.2. Bản thư viện 23
4.3.3. Bản tự cài đặt bằng NumPy 23
4.3.4. Kiểm chứng 24
4.4. Mô hình 2: Random Forest 25
4.4.1. Cơ sở lý thuyết 25
4.4.2. Bản thư viện 26
4.4.3. Bản tự cài đặt bằng NumPy 26
4.4.4. Kiểm chứng thành phần: một cây 27
4.4.5. Tinh chỉnh bằng OOB và chọn số cây 27
4.5. Kỹ thuật xử lý mất cân bằng 28
CHƯƠNG 5. KẾT QUẢ VÀ ĐÁNH GIÁ 29
5.1. Kết quả tổng thể trên tập test 29
5.2. Sai số theo phân khúc giá 29
5.3. Dự đoán so với giá thật và phần dư 30
5.4. Kiểm định ý nghĩa thống kê bằng bootstrap 31
5.5. Bản tự cài đặt so với bản thư viện 32
5.6. Diễn giải mô hình 33
5.6.1. Hệ số của Ridge 33
5.6.2. Độ quan trọng đặc trưng của Random Forest 33
5.6.3. Đường khấu hao 35
5.7. Thí nghiệm log giá và xử lý mất cân bằng 35
5.8. Ảnh hưởng của tin đăng lại 36
5.9. Lưu và sử dụng mô hình 37
CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 38
6.1. Kết quả đạt được 38
6.2. Hạn chế 38
6.3. Hướng phát triển 39
TÀI LIỆU THAM KHẢO 40

PHỤ LỤC 41
A. Cấu trúc mã nguồn.....41
B. Cách chạy.....41
C. Phân công công việc 41

DANH MỤC HÌNH

Hình 3.1. Mười hãng xe có nhiều tin đăng nhất (dữ liệu thô) 12

Hình 3.2. Phân bố giá rao bán trên thang gốc và thang log.....12

Hình 3.3. Giá rao theo năm sản xuất.....13

Hình 3.4. Phân bố giá theo hãng xe 13

Hình 3.5. Quan hệ giữa số km đã đi và giá (mật độ tin đăng).....14

Hình 3.6. Tỉ lệ giá trị thiếu theo cột (dữ liệu thô).....17

Hình 4.1. Chia train/test phân tầng: tỉ lệ các phân khúc giá như nhau ở hai tập 21

Hình 4.2. Chọn α cho Ridge bằng 5-fold cross-validation 23

Hình 4.3. Kiểm chứng Ridge: đường CV tự viết trùng thư viện và quá trình hội tụ của gradient descent 25

Hình 4.4. OOB RMSE của rừng tự cài đặt theo số cây 28

Hình 5.1. So sánh MAPE và tỉ lệ xe được định giá sát giữa các mô hình.....29

Hình 5.2. MAPE theo phân khúc giá 30

Hình 5.3. Giá dự đoán so với giá thật trên tập test (thang log).....31

Hình 5.4. Phân bố phần dư và sai số tuyệt đối theo phân khúc 31

Hình 5.5. Khoảng tin cậy 95% của MAPE và của hiệu MAPE.....32

Hình 5.6. Dự đoán của bản tự cài đặt so với bản thư viện trên từng xe trong tập test .32

Hình 5.7. Các hệ số Ridge ảnh hưởng mạnh nhất 33

Hình 5.8. Độ quan trọng đặc trưng: bản thư viện và bản tự cài đặt.....34

Hình 5.9. Giá dự đoán theo tuổi xe của ba mẫu xe ở ba phân khúc 35

Hình 5.10. MAPE theo phân khúc của các phương án (CV trên train) 36

DANH MỤC BẢNG

Bảng 1.1. Các bước thực hiện đề tài 9
Bảng 2.1. Từ điển dữ liệu thô 10
Bảng 2.2. Cơ chế thu thập dữ liệu và cách xử lý tương ứng.....11
Bảng 3.1. Các giá trị giả trong dữ liệu thô.....14
Bảng 3.2. Quy tắc phát hiện giá trị bất hợp lý 15
Bảng 3.3. Ví dụ tin bị loại vì nghi ghi giá trả trước / đặt cọc 16
Bảng 3.4. Cơ chế thiếu và cách điền cho từng cột.....17
Bảng 3.5. Quy tắc gộp nhãn hiếm theo cấp 18
Bảng 3.6. So sánh các cách gộp nhãn (5-fold CV trên train, MAPE) 19
Bảng 3.7. Phân bố dữ liệu theo phân khúc giá.....19
Bảng 3.8. Số dòng qua từng bước tiền xử lý.....20
Bảng 4.1. Đặc trưng đầu vào của hai mô hình.....21
Bảng 4.2. Các thước đo đánh giá 22
Bảng 4.3. Kiểm chứng Ridge tự cài đặt với bản thư viện.....24
Bảng 4.4. Siêu tham số của Random Forest 26
Bảng 4.5. Năm cấu hình Random Forest tốt nhất (thư viện) 26
Bảng 4.6. Các lớp của Random Forest tự cài đặt.....26
Bảng 4.7. Kiểm chứng cây hồi quy tự cài đặt.....27
Bảng 4.8. Tinh chỉnh Random Forest tự cài đặt bằng OOB (30 cây).....27
Bảng 4.9. Các kỹ thuật xử lý mất cân bằng được thử nghiệm.....28
Bảng 5.1. Kết quả trên tập test.....29
Bảng 5.2. MAPE theo phân khúc giá trên tập test (TV: thư viện, TCĐ: tự cài đặt).....29
Bảng 5.3. Hiệu MAPE giữa các mô hình (điểm %), bootstrap ghép cặp 31
Bảng 5.4. Tóm tắt so sánh bản tự cài đặt và bản thư viện 32
Bảng 5.5. Độ quan trọng đặc trưng của Random Forest (gộp theo đặc trưng gốc) 34
Bảng 5.6. Thí nghiệm trên Random Forest (5-fold CV trên tập train) 35
Bảng 5.7. Sai số trên nhóm tin có và không có bản sao trong tập train.....37
Bảng 5.8. Các file mô hình đã lưu 37

Bảng 5.9. Ví dụ dự đoán giá (triệu đồng) 37
Bảng PL.1. Cấu trúc thư mục dự án 41
Bảng PL.2. Phân công công việc trong nhóm 41

DANH MỤC TỪ VIẾT TẮT VÀ THUẬT NGỮ

Từ viết tắt	Tiếng Anh	Ý nghĩa
MAPE	Mean Absolute Percentage Error	Sai số phần trăm tuyệt đối trung bình (thước đo chính)
MedAPE	Median Absolute Percentage Error	Trung vị sai số phần trăm tuyệt đối
MAE	Mean Absolute Error	Sai số tuyệt đối trung bình (triệu đồng)
RMSE	Root Mean Squared Error	Căn bậc hai sai số bình phương trung bình
R^2	Coefficient of determination	Hệ số xác định
CV	Cross-validation	Kiểm định chéo (K-fold)
OOB	Out-of-bag	Các mẫu không được chọn vào mẫu bootstrap của một cây
CART	Classification and Regression Tree	Cây phân loại và hồi quy
RF	Random Forest	Rừng ngẫu nhiên
SSE	Sum of Squared Errors	Tổng bình phương sai số
KTC	Confidence interval	Khoảng tin cậy
EDA	Exploratory Data Analysis	Phân tích khám phá dữ liệu

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

1.1. Bối cảnh

Thị trường ô tô cũ ở Việt Nam ngày càng sôi động, và phần lớn giao dịch bắt đầu từ các trang rao vặt trực tuyến như Chợ Tốt (`xe.chotot.com`). Người bán thường tự đặt giá theo cảm tính hoặc tham khảo vài tin tương tự, còn người mua khó biết một mức giá rao là hợp lý hay không. Một mô hình ước lượng giá từ các thông tin cơ bản của xe (hãng, dòng xe, năm sản xuất, số km, hộp số, nhiên liệu, kiểu dáng...) giúp cả hai phía có một mốc tham chiếu khách quan.

Đề tài được thực hiện theo yêu cầu của môn Khoa học dữ liệu: xây dựng **một mô hình học máy từ đầu đến cuối**, từ thu thập dữ liệu, tiền xử lý, huấn luyện đến đánh giá bằng các thước đo phù hợp, có áp dụng **kỹ thuật xử lý dữ liệu mất cân bằng**. Các mô hình phải được **tự cài đặt** thuật toán, không chỉ gọi thư viện.

1.2. Phát biểu bài toán

Đây là bài toán **hồi quy có giám sát**:

- **Đầu vào:** các thuộc tính của một tin rao bán ô tô: hãng, dòng xe, xuất xứ, tình trạng (mới / đã sử dụng), hộp số, nhiên liệu, kiểu dáng, số chỗ, năm sản xuất, số km đã đi.
- **Đầu ra:** giá rao bán dự đoán (triệu đồng).
- **Dữ liệu:** 14.835 tin rao bán ô tô thu thập từ Chợ Tốt ngày 25/10/2026; sau tiền xử lý còn 13.937 tin.
- **Thước đo chính:** MAPE (sai số phần trăm trung bình), báo cáo kèm theo từng phân khúc giá.

1.3. Mục tiêu và phạm vi

1. Thu thập dữ liệu tin rao bán ô tô từ Chợ Tốt bằng web scraping.
2. Làm sạch dữ liệu: xử lý giá trị thiếu, giá trị bất hợp lý, giá giả, giá trả trước, tin trùng; gộp nhãn hiếm; tạo đặc trưng.
3. Xây dựng hai mô hình học máy cổ điển: **Ridge Regression** (tuyến tính) và **Random Forest** (phi tuyến). Mỗi mô hình có hai bản: dùng thư viện `scikit-learn` và **tự cài đặt bằng NumPy**, rồi kiểm chứng hai bản cho kết quả tương đương.
4. Đánh giá bằng nhiều thước đo, kiểm định ý nghĩa thống kê của chênh lệch giữa các mô hình.

5. Thử nghiệm các kỹ thuật xử lý mất cân bằng (trọng số mẫu, oversampling, undersampling) và đánh giá tác động.
6. Diễn giải mô hình: yếu tố nào quyết định giá xe, xe mất giá theo tuổi như thế nào.

Phạm vi: giá trong dữ liệu là **giá rao bán**, không phải giá giao dịch thực tế. Dữ liệu là ảnh chụp thị trường tại một thời điểm, không xét biến động giá theo thời gian.

1.4. Quy trình thực hiện

Đề tài đi theo quy trình chuẩn của một dự án khoa học dữ liệu, mỗi bước tương ứng với một notebook:

Bảng 1.1. *Các bước thực hiện đề tài*

Bước	Nội dung	File	Kết quả
1. Thu thập	Crawl 750 trang danh sách tin, đọc thuộc tính từng tin	`crawl_data.ipynb`	`raw_data_oto.csv` (14.835 dòng, 14 cột)
2. Khám phá	Biểu đồ phân bố, tương quan, giá trị thiếu	`visualize_raw_data.ipynb`	Biểu đồ trong `figures/`
3. Tiền xử lý	Làm sạch, điền thiếu, gộp nhãn hiếm, tạo đặc trưng	`preprocess_data.ipynb`	`clean_data_oto.csv` (13.937 dòng, 18 cột)
4. Mô hình	Ridge và Random Forest (thư viện + tự cài đặt), đánh giá, thí nghiệm	`model.ipynb`	Mô hình đã lưu trong `model/`
5. Triển khai	Nạp mô hình, dự đoán từ thông tin thô của xe	`model/price_model.py`	Hàm `predict_price()`

1.5. Công cụ sử dụng

- **Ngôn ngữ:** Python 3, môi trường Jupyter Notebook.
- **Thu thập dữ liệu:** `requests`, `BeautifulSoup`.
- **Xử lý và trực quan hóa:** `pandas` 3.0, `NumPy`, `matplotlib`.
- **Học máy:** `scikit-learn` 1.9 (bản thư viện, dùng làm chuẩn đối chiếu); `NumPy` (bản tự cài đặt); `joblib` để lưu mô hình.

CHƯƠNG 2. THU THẬP DỮ LIỆU

2.1. Nguồn dữ liệu và phương pháp thu thập

Dữ liệu được thu thập từ chuyên mục mua bán ô tô của Chợ Tốt (`https://xe.chotot.com/mua-ban-oto`) bằng notebook `crawl_data.ipynb`, hoàn tất ngày **25/10/2026**. Quy trình gồm hai tầng:

1. **Duyệt trang danh sách:** lần lượt truy cập 750 trang danh sách tin mới nhất (`?page=1` đến `?page=750`), lấy đường dẫn của từng tin.
2. **Đọc trang chi tiết:** với mỗi tin, tải trang và đọc các thẻ HTML có thuộc tính `itemprop` tương ứng với từng thuộc tính của xe (riêng giá đọc từ thẻ `name="price"`). Kết quả được ghi nối tiếp vào `raw_data_oto.csv`.

Code có cơ chế **chạy tiếp**: các URL đã có trong file được bỏ qua, nên nếu bị ngắt giữa chừng chỉ cần chạy lại. Các tin "ưu tiên" lặp lại ở đầu mọi trang cũng chỉ được ghi một lần. Kết quả là **14.835 tin** với 14 cột (số thứ tự, URL và 12 thuộc tính).

2.2. Thu thập có trách nhiệm

- Chỉ lấy thông tin công khai trên trang tin rao, không đăng nhập, không lấy thông tin liên hệ của người bán; dữ liệu chỉ dùng cho mục đích học tập.
- Nghỉ giữa các yêu cầu (0,5 giây giữa các tin, 1 giây giữa các trang) và tự lùi thời gian khi máy chủ trả về mã 403/429, để không gây tải cho máy chủ.
- Hạn chế kỹ thuật: danh sách tin được tìm bằng một class CSS sinh tự động; khi trang đổi giao diện cần sửa lại bộ chọn.

2.3. Từ điển dữ liệu

Bảng 2.1 mô tả các cột thu được. Cột *"Người bán nhập thế nào"* rất quan trọng cho bước tiền xử lý: cột **chọn từ danh sách** có tập giá trị cố định, viết thống nhất; còn cột **gõ tự do** (số km, giá) là nguồn lỗi nhập liệu chính.

Bảng 2.1. *Từ điển dữ liệu thô*

Cột	Ý nghĩa	Người bán nhập	Sau xử lý
`mileage_v2`	Số km đã đi	**gõ tự do** (dạng 109.366)	số (`mileage_km`)
`carorigin`	Xuất xứ	chọn từ danh sách, không bắt buộc	phân loại
`condition_ad`	Tình trạng (Mới / Đã sử dụng)	chọn từ danh sách	phân loại
`veh_warranty_policy`	Chính sách bảo hành	(một giá trị duy nhất)	bỏ
`carbrand`	Hãng xe	chọn từ danh sách	phân loại

Cột	Ý nghĩa	Người bán nhập	Sau xử lý
`carmodel`	Dòng xe	chọn theo hãng	phân loại
`mfdate`	Năm sản xuất	chọn (có mục "trước năm 1980")	số
`gearbox`	Hộp số	chọn (3 giá trị)	phân loại
`fuel`	Nhiên liệu	chọn (4 giá trị)	phân loại
`cartype`	Kiểu dáng	chọn (9 giá trị), không bắt buộc	phân loại
`carseats`	Số chỗ ngồi	chọn (có mục "Khác"), không bắt buộc	số
`price`	Giá rao bán (đồng)	**gõ tự do**	biến mục tiêu
`url`	Đường dẫn tin	sinh tự động, chứa quận/huyện và tỉnh	trích `province`, rồi bỏ

2.4. Cơ chế thu thập và hệ quả cho tiền xử lý

Hiểu **cách dữ liệu được tạo ra** giúp biết mỗi loại lỗi đến từ đâu và nên xử lý thế nào:

Bảng 2.2. *Cơ chế thu thập dữ liệu và cách xử lý tương ứng*

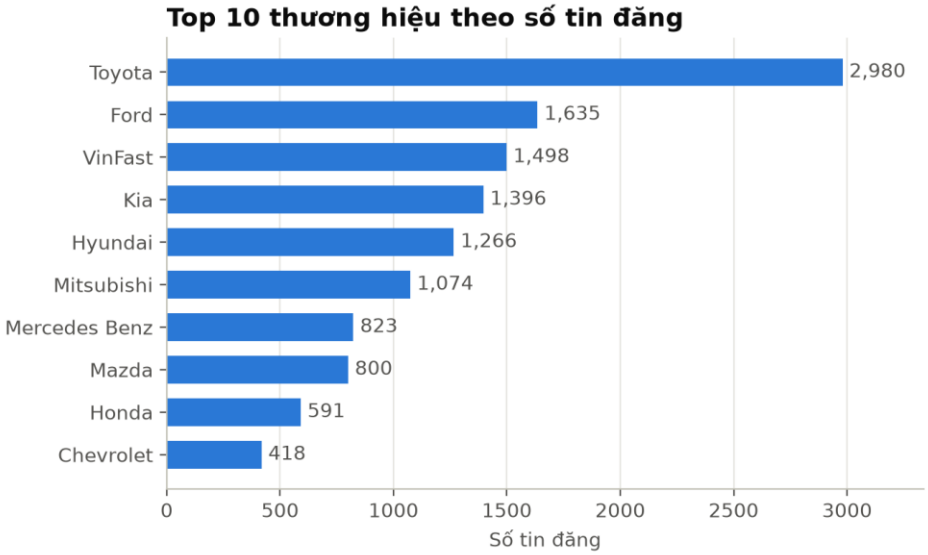
Cơ chế	Hệ quả	Cách xử lý
Không tìm thấy thẻ của một thuộc tính → ghi chuỗi `"N/A"`	`N/A` là thiếu thật; pandas mặc định tự đổi thành NaN, làm mất dấu vết	Đọc với `keep_default_na=False`, rồi tự đổi về NaN
Trang hiển thị `"Đang cập nhật"` khi người bán không chọn xuất xứ	Cũng là thiếu thật	Đổi về NaN
Danh sách có sẵn mục `"Khác"`, `"Dòng khác"`, `"Nước khác"`	Là lựa chọn thật của người bán	Giữ làm nhãn (trừ `carseats = "Khác"`)
Số km và giá là ô gõ tự do	Gõ thiếu/thừa số 0, ghi tiền trả trước, giá giả 123.456.789	Quy tắc kiểm tra ở mục 3.4, 3.5
Xe "Mới" không có ô số km	Số km thiếu ở mọi xe mới	Điền 0 km
Bỏ qua URL đã gặp	Mỗi dòng là một tin khác nhau; dòng trùng nội dung là tin đăng lại	Loại trùng theo nội dung
URL dạng `mua-ban-oto-<quận>-<tỉnh>/<mã>.htm`	Có thông tin vị trí	Trích tỉnh/thành
Một lần thu thập, 750 trang mới nhất	Ảnh chụp thị trường tại một thời điểm; giá là giá rao	Tuổi xe tính theo năm 2026

CHƯƠNG 3. KHÁM PHÁ VÀ TIỀN XỬ LÝ DỮ LIỆU

Chương này trình bày các bước biến dữ liệu thô thành tập dữ liệu sạch cho mô hình (notebook `preprocess_data.ipynb`). Mỗi quyết định đều kèm lý do, và đều xuất phát từ việc **giá là biến cần dự đoán**.

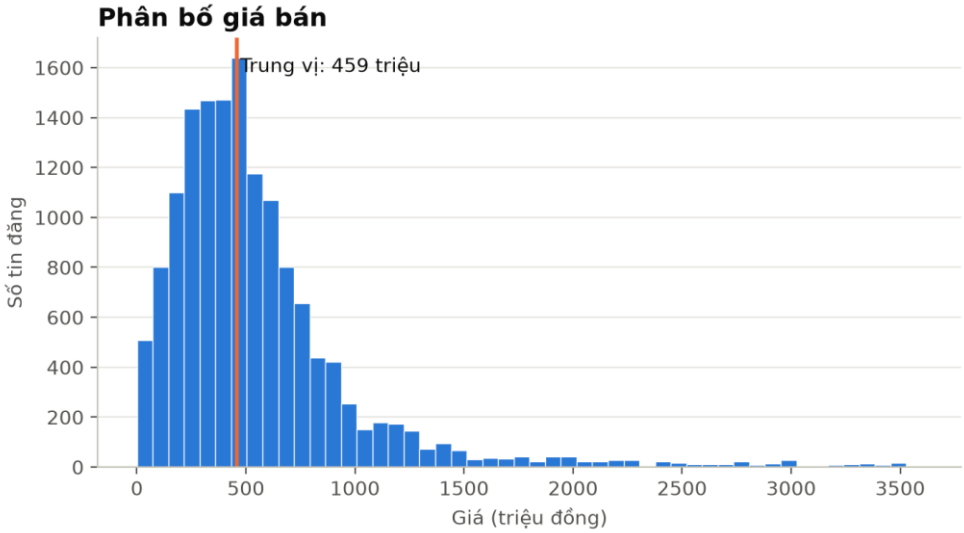
3.1. Khám phá dữ liệu ban đầu

Dữ liệu thô có 14.835 dòng và 14 cột. Thị trường tập trung vào các hãng phổ thông: Toyota chiếm khoảng 20% số tin, tiếp theo là Ford, VinFast, Kia, Hyundai (Hình 3.1).



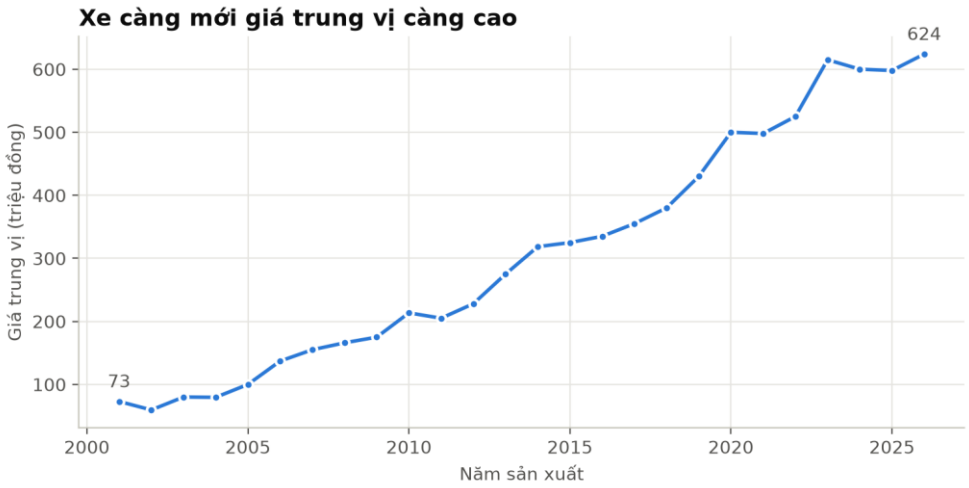
Hình 3.1. *Mười hãng xe có nhiều tin đăng nhất (dữ liệu thô)*

Giá rao **lệch phải rất mạnh** (độ lệch skewness = 5,08): đa số xe có giá 200-800 triệu, nhưng có một đuôi dài các xe vài tỷ đến hàng chục tỷ đồng (Hình 3.2). Đặc điểm này ảnh hưởng trực tiếp đến cách chọn biến mục tiêu (mục 3.7).

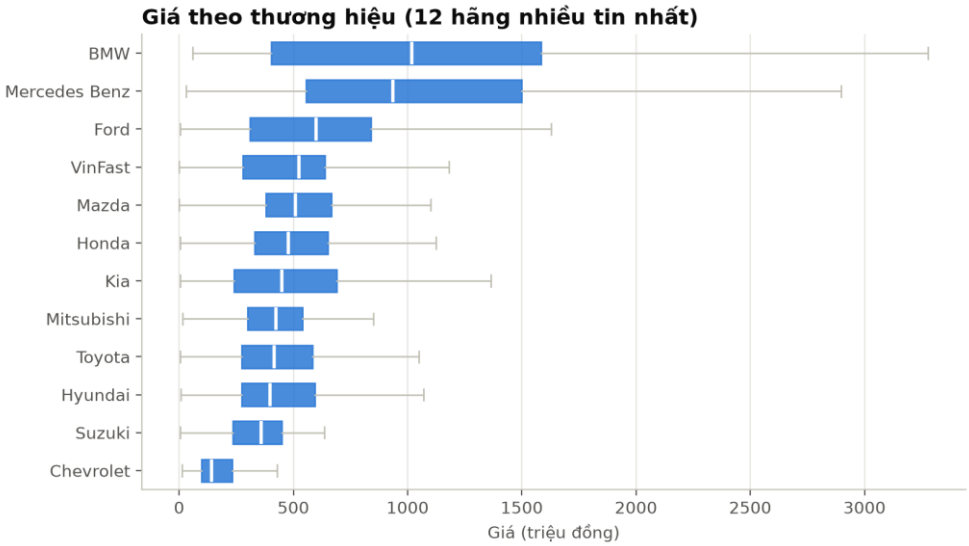


Hình 3.2. *Phân bố giá rao bán trên thang gốc và thang log*

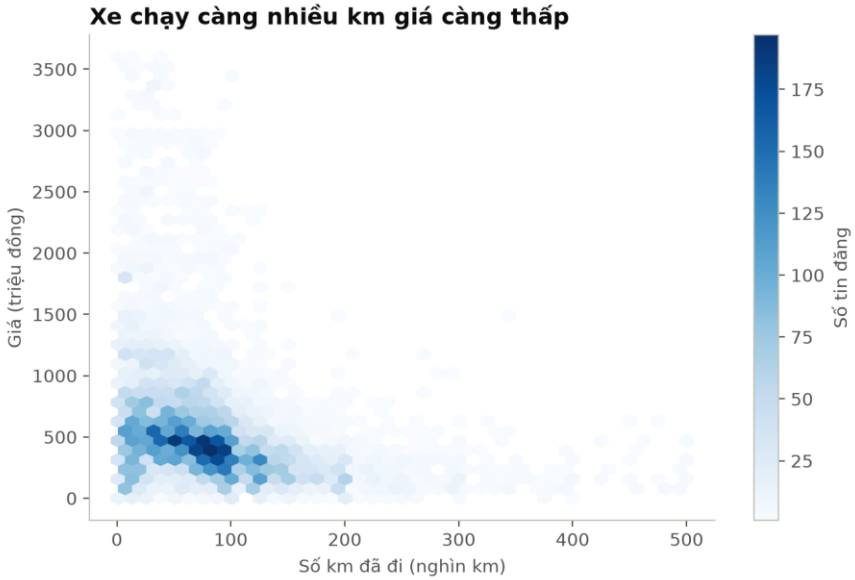
Hai yếu tố có quan hệ rõ nhất với giá là **năm sản xuất** và **số km**: xe càng mới giá càng cao, xe chạy càng nhiều km giá càng thấp, và quan hệ không tuyến tính (xe mất giá nhanh trong những năm đầu). Giá cũng chênh lệch lớn giữa các hãng.



Hình 3.3. *Giá rao theo năm sản xuất*



Hình 3.4. *Phân bố giá theo hãng xe*



Hình 3.5. *Quan hệ giữa số km đã đi và giá (mật độ tin đăng)*

3.2. Chuẩn hóa kiểu dữ liệu và giá trị giả

Bảng 3.1 thống kê các giá trị "giả" trong dữ liệu thô, tức các chuỗi không phải giá trị thật của thuộc tính:

Bảng 3.1. *Các giá trị giả trong dữ liệu thô*

Cột	`N/A` (không có thẻ)	`Đang cập nhật`	`Khác`	Số giá trị khác nhau
mileage_v2	1.811	0	0	1.995
carorigin	0	6.012	0	11
veh_warranty_policy	0	0	0	1
cartype	4.883	0	0	10
carseats	3.028	0	659	12
price	1	0	0	1.433

- `N/A` và `Đang cập nhật` đều là người đăng **không điền thông tin**, nên được đổi thành giá trị thiếu (NaN). Nếu để nguyên, mô hình sẽ coi "Đang cập nhật" là một xuất xứ thật.
- `Khác` trong `carseats` không đổi ra số được nên coi là thiếu; còn `Dòng khác`, `Kiểu dáng khác` là nhãn có nghĩa nên giữ nguyên.
- `price`, `mileage_v2`, `carseats` dạng chuỗi `1.095.000.000` được bỏ dấu chấm rồi ép sang số; giá đổi sang đơn vị triệu đồng (`price_tr`).
- `mfdate = "trước năm 1980"` (12 dòng) được gán 1979.

3.3. Trích vị trí, loại cột thừa và tin trùng

Vị trí từ URL. Mọi URL có dạng `mua-ban-oto-<quận/huyện>-<tỉnh>/<mã tin>.htm`. So khớp phần cuối với danh sách 63 tỉnh/thành để lấy cột `province` (giữ dạng slug của Chợ Tốt như `tp-ho-chi-minh`, `ha-noi`). Trích được tỉnh cho 100% số tin. Không tách quận/huyện vì có 453 quận/huyện, trung vị chỉ 4 tin mỗi quận, 315 quận dưới 10 tin: dữ liệu quá thưa. Ngay cả ở mức tỉnh, TP.HCM và Hà Nội đã chiếm 67% số tin, 28 tỉnh có dưới 30 tin.

Cột loại bỏ: `no` và `url` là định danh, không mang thông tin dự đoán; `veh_warranty_policy` chỉ có một giá trị duy nhất ("Bảo hành hãng") ở mọi dòng.

Tin trùng: 691 dòng (4,7%) giống hệt nhau ở mọi thuộc tính và giá (331 trong số đó là xe mới, thường do đại lý đăng nhiều xe cùng cấu hình). Giữ lại sẽ làm lệch phân phối, và khi chia train/test thì cùng một xe có thể nằm ở cả hai phía, làm điểm đánh giá cao ảo. Chỉ giữ một bản, còn lại **14.144 dòng**.

3.4. Giá trị bất hợp lý

Bước này làm **trước** khi điền thiếu, để không điền dựa trên số liệu sai. Phân biệt hai loại: **sai dữ liệu** (nhập nhầm, không thể xảy ra) thì loại hoặc sửa; **ngoại lệ thật** (xe siêu sang giá chục tỷ) thì giữ, vì đó là xe có thật.

Bảng 3.2. *Quy tắc phát hiện giá trị bất hợp lý*

Cột	Quy tắc	Lý do	Số dòng
Giá	Loại nếu < 20 triệu hoặc > 10 tỷ	Gõ thiếu/thừa số 0; không biết giá đúng nên loại	64
Giá	Loại giá là dãy số liên tiếp (123.456.789...)	Giá giả khi người bán không muốn công khai giá	14
Tình trạng	"Mới" nhưng hơn 3 năm tuổi → "Đã sử dụng"	Ghi sai tình trạng (ví dụ Tucson 2011 "mới")	6 (sửa)
Số km	Xe đã dùng > 500.000 km → thiếu	Nhập thừa số 0	106
Số km	Xe ≥ 2 năm tuổi mà < 100 km → thiếu	Nhập theo đơn vị nghìn km	447
Số km	> 100.000 km/năm → thiếu	Bất khả thi với xe dân dụng	25

Các giá như 399.999.999 hay 888.888.888 vẫn được **giữ**, vì đó là cách định giá tâm lý hoặc chọn số đẹp, vẫn là giá thật. Sau bước này còn **14.066 dòng** (loại 78 dòng); 578 giá trị số km bất hợp lý được chuyển thành thiếu để điền lại ở mục 3.6.

3.5. Giá trả trước / đặt cọc

Ngưỡng 20 triệu chỉ bắt được lỗi gõ thiếu số. Vẫn còn những tin mà "giá" thực ra là **số tiền trả trước hoặc đặt cọc**, ví dụ xe đời 2025-2026 còn mới mà giá chỉ 30-100 triệu. Một ngưỡng tuyệt đối không bắt được trường hợp này, vì 100 triệu hợp lý với xe 20 năm tuổi nhưng vô lý với xe 1 năm tuổi.

Quy tắc: loại tin có giá **thấp hơn 30% giá trung vị của các xe cùng hãng, dòng và năm sản xuất** (chỉ xét nhóm có ít nhất 3 xe). Không xe nào mất 70% giá so với xe cùng đời chỉ vì tình trạng hay số km. Kết quả: loại **129 tin**, 94% trong số đó có giá dưới 200 triệu; còn lại **13.937 dòng**.

Bảng 3.3. *Ví dụ tin bị loại vì nghi ghi giá trả trước / đặt cọc*

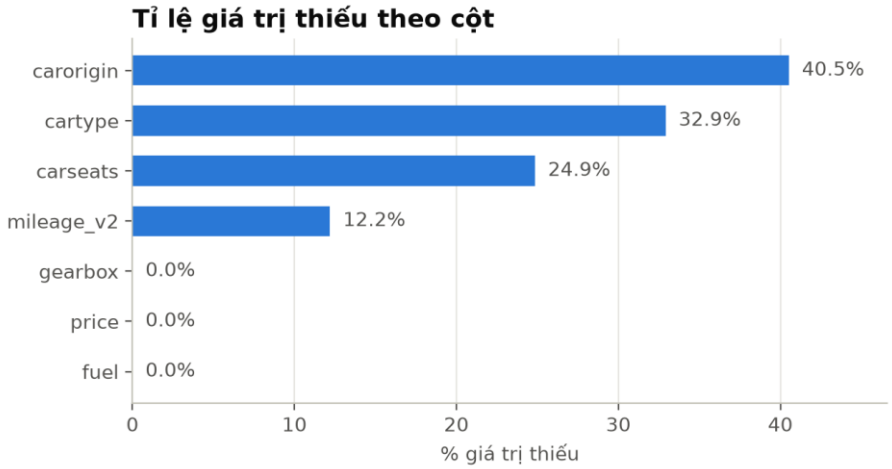
Xe	Năm	Tình trạng	Giá rao (triệu)	Trung vị nhóm (triệu)
Toyota Camry	2026	Đã sử dụng	300	1.400
Ford Everest	2026	Đã sử dụng	320	1.325
Toyota Fortuner	2026	Mới	125	1.249
Hyundai Santa Fe	2025	Mới	220	959
Mercedes Benz E Class	2019	Đã sử dụng	156	889

Đây là bước **sửa lỗi nhãn**: quy tắc dùng giá trên toàn bộ dữ liệu nhưng chỉ để loại dòng có đáp án sai, không đưa thông tin giá vào đặc trưng nên không gây rò rỉ dữ liệu vào đầu vào của mô hình.

Ngoại lệ theo IQR của log giá còn 696 dòng (5,0%), chủ yếu là xe sang thật (Mercedes, Lexus, Bentley, Range Rover...) hoặc xe cổ rất rẻ. Đây là thông tin thật của thị trường nên **giữ nguyên**; ảnh hưởng của chúng được giảm bằng biến đổi log.

3.6. Xử lý giá trị thiếu

Trước khi điền cần hiểu **vì sao thiếu**, vì cách điền phụ thuộc cơ chế thiếu.



Hình 3.6. *Tỉ lệ giá trị thiếu theo cột (dữ liệu thô)*

Bảng 3.4. *Cơ chế thiếu và cách điền cho từng cột*

Cột	Cơ chế thiếu	Cách xử lý
`mileage_km`	**Toàn bộ** 1.235 xe mới thiếu km (trang tin xe mới không có ô số km); chỉ 244 xe đã dùng thiếu thật, cộng 578 giá trị bất hợp lý	Xe mới: **0 km**. Xe đã dùng: trung vị theo năm sản xuất (814 xe, 5,8%), đánh dấu `mileage_missing = 1`
`carorigin`	Thiếu ~41%; một mẫu xe có thể vừa nhập khẩu vừa lắp ráp	Nhãn riêng **"Không rõ"** (5.787 dòng)
`cartype`	Thiếu ~33%; kiểu dáng gần như do dòng xe quyết định	Mode theo hãng + dòng xe (4.336); còn lại "Không rõ" (243)
`carseats`	Thiếu ~25%; số chỗ do dòng xe quyết định	Mode theo dòng xe (3.352) → theo hãng (183) → mode chung (4)
`price`	1 dòng	**Xóa** (không bịa biến mục tiêu)

Vì sao không điền trung vị số km cho xe mới? Điền trung vị sẽ biến xe mới thành "xe đã chạy 66.000 km", sai lệch nghiêm trọng. Đây là thiếu **có cấu trúc** (không đo vì không có ý nghĩa), giá trị đúng là 0.

Cột đánh dấu `mileage_missing`: giá trị điền cho xe đã dùng chỉ là "đoán trung bình". Cột này cho mô hình biết dòng nào là ước lượng để tự học mức độ tin cậy; việc người bán không khai số km cũng có thể là một tín hiệu về giá.

Vì sao `cartype` không lùi về cấp hãng còn `carseats` thì có? Trong một dòng xe, kiểu dáng gần như cố định; nhưng một hãng bán đủ loại (Ford có Ranger bán tải, Everest SUV, Focus sedan), nên gán kiểu dáng phổ biến của hãng là đoán mò. Số chỗ thì đa số là 5, đoán sai theo hãng thường chỉ lệch 5 ↔ 7 chỗ.

3.7. Biến đổi log cho giá

Giá lệch phải mạnh khiến mô hình hồi quy bị kéo bởi vài xe rất đắt. Lấy logarit đưa phân phối về gần đối xứng (độ lệch giảm từ **5,08 xuống −0,20**) và biến chênh lệch tuyệt đối thành chênh lệch **tương đối**: xe 100 → 150 triệu và xe 1 → 1,5 tỷ đều là tăng 50%, khớp với cách người mua nghĩ về giá và với thước đo MAPE.

$$y = \ln(p)$$

$$\hat{p} = e^{\hat{y}}$$

Với p là giá (triệu đồng), mô hình học `price_log`, dự đoán xong dùng `exp()` để đổi về triệu đồng; **mọi thước đo đều tính trên giá thật**. Lựa chọn này được kiểm chứng bằng thí nghiệm ở mục 5.7.

3.8. Gộp nhãn hiếm theo cấp

Dữ liệu có 69 hãng và 509 tổ hợp hãng + dòng xe, nhiều nhãn chỉ có 1-5 tin. One-hot nguyên vẹn sinh hàng trăm cột thưa, mô hình học quy luật từ vài mẫu là học nhiễu. Cách gộp đơn giản (mọi dòng hiếm thành một nhãn `"Dòng khác"`) có vấn đề: nhãn này chiếm 17% dữ liệu nhưng tới **65% số xe trên 2 tỷ**, một chiếc Mercedes G63 và một chiếc Kia đời cũ mang cùng nhãn.

Đề tài dùng cách **gộp theo cấp**: mỗi xe lùi về nhóm gần nhất còn đủ mẫu.

Bảng 3.5. *Quy tắc gộp nhãn hiếm theo cấp*

Cấp	Quy tắc	Ví dụ
Dòng xe	Giữ `Hãng + Dòng` nếu có ≥ 10 tin	Toyota Vios
→ Hãng	Dòng hiếm → `"<Hãng> - dòng khác"`	Mercedes Benz - dòng khác
Hãng	Giữ hãng nếu có ≥ 30 tin	Volvo, Subaru
→ Nhóm hãng	Hãng hiếm → `"Hãng cao cấp khác"` / `"Hãng phổ thông khác"`	Bentley → cao cấp; Wuling → phổ thông

Nhóm cao cấp / phổ thông được phân theo **định vị thương hiệu** (hiểu biết thị trường), **không** dựa vào giá trong dữ liệu, để không dùng biến mục tiêu khi tạo đặc trưng. Kết quả: 69 hãng → **28 nhóm**, 509 dòng → **214 nhóm**; 8,9% số xe mang nhãn "... - dòng khác". Nhóm "Hãng cao cấp khác" có 93 tin từ 13 hãng (giá trung vị 1 tỷ), "Hãng phổ thông khác" có 188 tin từ 30 hãng (giá trung vị 390 triệu).

Cách gộp và ngưỡng được **chọn bằng 5-fold CV chỉ trên tập train** (Bảng 3.6). Gộp theo cấp cải thiện rõ Ridge (MAPE 21,9% → 18,9%; xe > 2 tỷ 43,0% → 32,2%) vì mô hình tuyến tính chỉ có một hệ số cho mỗi nhãn. Random Forest được lợi ít hơn vì tự bù thông tin bằng các đặc trưng khác. Không phương án nào làm mô hình xấu đi.

Bảng 3.6. *So sánh các cách gộp nhãn (5-fold CV trên train, MAPE)*

Phương án	Ridge: MAPE	< 200 tr	> 2 tỷ	RF: MAPE	< 200 tr	> 2 tỷ
Một nhãn chung (50 / 30)	0,219	0,522	0,430	0,163	0,374	0,292
Gộp theo cấp (50 / 30)	0,206	0,474	0,392	0,163	0,361	0,278
Gộp theo cấp (30 / 10) – chọn	**0,189**	**0,449**	**0,322**	**0,161**	**0,359**	**0,287**

3.9. Tạo đặc trưng

- `age` = 2026 − năm sản xuất: tuổi xe có ý nghĩa trực tiếp hơn năm tuyệt đối.
- `km_per_year` = km / max(tuổi, 1): xe 5 năm đi 150.000 km (nhiều khả năng chạy dịch vụ) khác hẳn xe 5 năm đi 50.000 km.
- `mileage_missing`: 1 nếu số km là giá trị điền.
- `province`: tỉnh/thành của tin đăng.

Vị trí có giúp dự đoán không? Thử thêm miền (3 nhóm) hoặc tỉnh/thành (15 nhóm, tỉnh dưới 100 tin gộp theo miền) bằng cùng cách CV: MAPE chỉ chênh ở chữ số thứ ba (Ridge 0,1895 / 0,1894 / 0,1897; Random Forest 0,1610 / 0,1604 / 0,1611), nhỏ hơn mức dao động giữa các lần chia fold. Giá rao chủ yếu do bản thân chiếc xe quyết định; chênh lệch giá giữa các miền chủ yếu do **cơ cấu xe khác nhau**, mà mô hình đã thấy qua các đặc trưng khác. Vì vậy `province` được giữ trong file để mô tả dữ liệu nhưng **không đưa vào mô hình**.

3.10. Dữ liệu mất cân bằng

Biến mục tiêu là số liên tục nên không có "lớp thiểu số" như bài toán phân loại. Mất cân bằng ở đây thể hiện ở **mật độ dữ liệu không đều theo vùng giá**: phần lớn xe ở phân khúc phổ thông, xe sang vài tỷ rất ít. Mô hình tối ưu sai số trung bình sẽ học tốt vùng đông mẫu và kém ở vùng hiếm. Để định lượng, giá được chia thành 5 phân khúc (chỉ dùng để phân tích, chia mẫu và đánh giá, không làm đặc trưng):

Bảng 3.7. *Phân bố dữ liệu theo phân khúc giá*

Phân khúc	Số tin	Tỉ lệ	Ít hơn phân khúc lớn nhất
< 200 triệu	1.972	14,1%	3,0 lần
200-500 triệu	5.907	42,4%	1,0 lần
500-1.000 triệu	4.529	32,5%	1,3 lần
1-2 tỷ	1.071	7,7%	5,5 lần
> 2 tỷ	458	3,3%	12,9 lần

Phân khúc > 2 tỷ ít hơn phân khúc lớn nhất gần 13 lần: mất cân bằng thật sự. Các đặc trưng cũng lệch: xe mới chỉ 8,5%, Hybrid 2,2%, hộp số bán tự động 0,7%. Các biện pháp áp dụng:

- **Chia train/test phân tầng theo phân khúc giá**, để tập test có đủ xe ở mọi phân khúc.
- **Học trên log giá**, giảm ảnh hưởng của đuôi phải.
- **Báo cáo MAPE theo từng phân khúc**, không chỉ một con số trung bình.
- **Thử trọng số mẫu, Random Oversampling, Random Undersampling** bằng CV trên tập train (mục 4.5 và 5.7).

File `clean_data_oto.csv` **chỉ chứa dữ liệu thật**: nếu ghi mẫu nhân bản vào file, người dùng sau chia ngẫu nhiên sẽ khiến bản sao của cùng một xe nằm ở cả train lẫn test.

3.11. Tổng kết tiền xử lý

Bảng 3.8. *Số dòng qua từng bước tiền xử lý*

Bước	Số dòng còn lại	Thay đổi
Dữ liệu thô	14.835	
Loại tin trùng nội dung	14.144	−691
Loại giá thiếu / < 20 triệu / > 10 tỷ / giá giả	14.066	−78
Loại giá trả trước / đặt cọc	13.937	−129
Dữ liệu sạch (giữ 93,9%)	**13.937**	**18 cột, không còn giá trị thiếu**

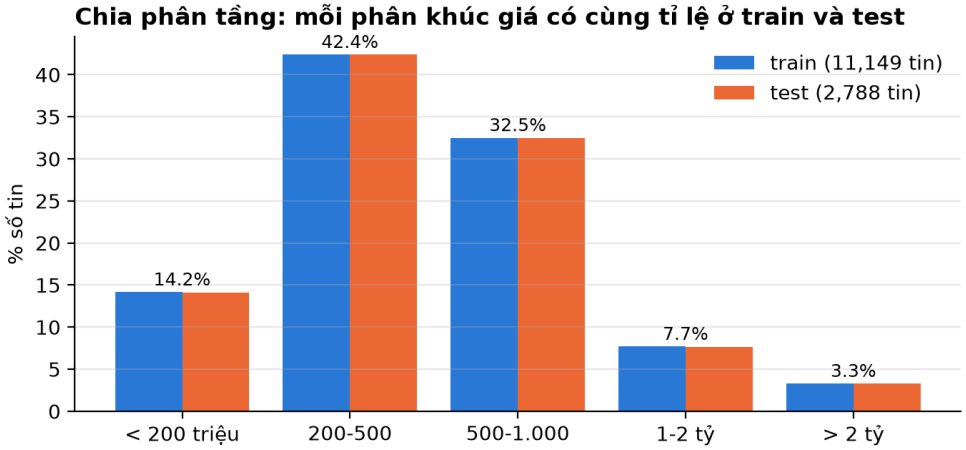
File `clean_data_oto.csv` gồm 18 cột: `carbrand`, `carbrand_grp`, `carmodel`, `carmodel_grp`, `carorigin`, `condition_ad`, `gearbox`, `fuel`, `cartype`, `carseats`, `mfdate`, `age`, `mileage_km`, `mileage_missing`, `km_per_year`, `province`, `price_tr`, `price_log`. Cột gốc `carbrand`, `carmodel` được giữ để tra cứu; mô hình dùng cột đã gộp `*_grp`.

CHƯƠNG 4. XÂY DỰNG MÔ HÌNH

4.1. Thiết kế thí nghiệm

4.1.1. Chia dữ liệu

Dữ liệu được chia **80% train (11.149 tin) / 20% test (2.788 tin)**, phân tầng theo phân khúc giá, `random_state = 42`. Không tách tập validation riêng: tham số được chọn bằng **5-fold cross-validation** (hoặc **OOB** với Random Forest) **trên tập train**, ổn định hơn một tập validation cố định khi xe trên 2 tỷ chỉ có khoảng 460 tin. Tập test chỉ dùng **một lần** để báo cáo kết quả cuối.



Hình 4.1. *Chia train/test phân tầng: tỉ lệ các phân khúc giá như nhau ở hai tập*

4.1.2. Đặc trưng cho từng mô hình

Hai mô hình dùng chung đặc trưng phân loại (one-hot), nhưng đặc trưng số được biến đổi theo bản chất mô hình:

Bảng 4.1. *Đặc trưng đầu vào của hai mô hình*

Nhóm	Đặc trưng	Ridge	Random Forest
Phân loại (one-hot)	hãng, dòng xe (đã gộp), xuất xứ, tình trạng, hộp số, nhiên liệu, kiểu dáng	✓	✓
Tuổi xe	`age`	`age`, `age²`	`age`
Số km	`mileage_km`, `km_per_year`	`log(1+km)`, `log(1+km/năm)`	giữ nguyên
Khác	`carseats`, `mileage_missing`	✓ (chuẩn hóa)	✓
Số cột sau mã hóa		278	277

- **Ridge cần `age²` và `log`** vì nó chỉ cộng các hiệu ứng tuyến tính, trong khi xe mất giá nhanh những năm đầu rồi chậm lại, và 10.000 km đầu "đắt" hơn 10.000 km sau.

- **Random Forest không cần**, vì cây chỉ so sánh ngưỡng nên mọi biến đổi đơn điệu cho cùng kết quả; thêm `age²` chỉ làm độ quan trọng đặc trưng bị chia đôi.
- **StandardScaler** chỉ cho Ridge, để khoản phạt L2 công bằng giữa các đặc trưng. Bộ mã hóa và chuẩn hóa **fit trên train** rồi áp lên test.
- **Không dùng:** `price_tr`, `price_log` (đáp án), `mfdate` (trùng `age`), `province` (không giúp dự đoán, mục 3.9).

4.2. Thước đo đánh giá

Tất cả thước đo tính trên **giá thật (triệu đồng)**, sau khi đổi dự đoán log về giá bằng `exp()`:

$$\text{MAPE} = \frac{1}{n}\sum_{i=1}^{n}\frac{|p_i - \hat{p}_i|}{p_i}$$

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|p_i - \hat{p}_i|$$

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(p_i - \hat{p}_i)^2}$$

trong đó p_i là giá thật và $\hat{p}_i$ là giá dự đoán của xe thứ i (triệu đồng).

Bảng 4.2. *Các thước đo đánh giá*

Thước đo	Ý nghĩa	Vì sao dùng
MAPE	Sai số phần trăm trung bình	**Thước đo chính**: lệch 100 triệu là nhỏ với xe 3 tỷ nhưng rất lớn với xe 150 triệu
MedAPE	Trung vị sai số phần trăm	Sai số "điển hình", không bị vài tin sai nặng kéo lệch
MAE	Sai số tuyệt đối trung bình (triệu)	Dễ hiểu: trung bình đoán lệch bao nhiêu tiền
RMSE	Căn trung bình bình phương sai số (triệu)	Phạt nặng lỗi lớn (thường ở xe sang)
R^2 (log)	Tỉ lệ biến thiên của log giá được giải thích	Đo trên đúng thang mô hình học
±10% / ±20%	Tỉ lệ tin đoán lệch không quá 10% / 20%	Diễn giải cho người dùng

Ngoài ra luôn báo **MAPE theo từng phân khúc giá**. Mô hình mốc (**baseline**) luôn đoán giá trung vị của tập train; mô hình nào cũng phải tốt hơn hẳn mức này mới có ý nghĩa.

4.3. Mô hình 1: Ridge Regression

4.3.1. Cơ sở lý thuyết

Ridge dự đoán log giá bằng tổ hợp tuyến tính của các đặc trưng, và tìm trọng số bằng cách tối thiểu hóa sai số bình phương **cộng một khoản phạt L2**:

$$\hat{y} = w_0 + \sum_{j=1}^{p} w_j x_j$$

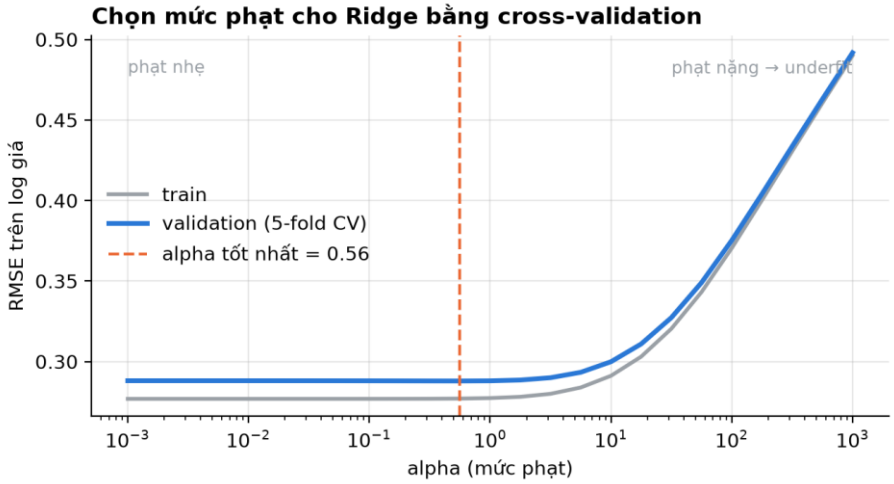
$$J(w) = \frac{1}{2n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2 + \frac{\alpha}{2}\sum_{j=1}^{p} w_j^2$$

Hệ số tự do `w₀` không bị phạt. **Vì sao cần khoản phạt:** sau one-hot có khoảng 270 cột, nhiều cột tương quan mạnh (hãng Toyota và dòng Toyota Vios luôn đi cùng nhau). Không có phạt, các hệ số có thể phình to và triệt tiêu lẫn nhau, mô hình nhạy với nhiễu. Siêu tham số α quá nhỏ thì dễ overfit, quá lớn thì các hệ số bị ép về 0 (underfit).

Diễn giải hệ số: vì mô hình dự đoán log giá, tăng x_j thêm 1 đơn vị thì giá nhân với $\exp(w_j)$, tức thay đổi khoảng $(\exp(w_j) - 1) \times 100\%$.

4.3.2. Bản thư viện

Dùng `sklearn.linear_model.Ridge`, tinh chỉnh α trên lưới 25 giá trị từ 10^{-3} đến 10^{3} bằng **GridSearchCV + 5-fold CV** (thước đo RMSE trên log giá). Kết quả: **α = 0,5623**, CV RMSE (log) = 0,2879, mất 8,9 giây.



Hình 4.2. *Chọn α cho Ridge bằng 5-fold cross-validation*

4.3.3. Bản tự cài đặt bằng NumPy

Lớp `RidgeScratch` tìm `w` bằng hai cách, đều tự viết:

(1) Nghiệm đóng (normal equation). Đạo hàm J(w) theo w và cho bằng 0, được hệ phương trình tuyến tính, giải trực tiếp bằng `np.linalg.solve` (không nghịch đảo ma trận để ổn định số):

$$\left(\frac{1}{n}A^TA+\alpha I'\right)w=\frac{1}{n}A^Ty$$

trong đó A là ma trận đặc trưng thêm cột 1 (cho w_0), I' là ma trận đơn vị nhưng phần tử ứng với w_0 bằng 0.

(2) Gradient Descent có momentum Nesterov. Bắt đầu từ w = 0, lặp lại bước ngược hướng gradient tại điểm "nhìn trước" v:

$$\nabla J(v)=\frac{1}{n}A^T(Av-y)+\alpha I'v$$

$$w_{t+1}=v_t-\eta\nabla J(v_t)$$

$$v_{t+1}=w_{t+1}+\frac{t-1}{t+2}(w_{t+1}-w_t)$$

Bước học η = 1/L với L là trị riêng lớn nhất của ma trận Hessian, đảm bảo hội tụ. Cách này dùng để minh họa quá trình tối ưu và kiểm chứng chéo với nghiệm đóng.

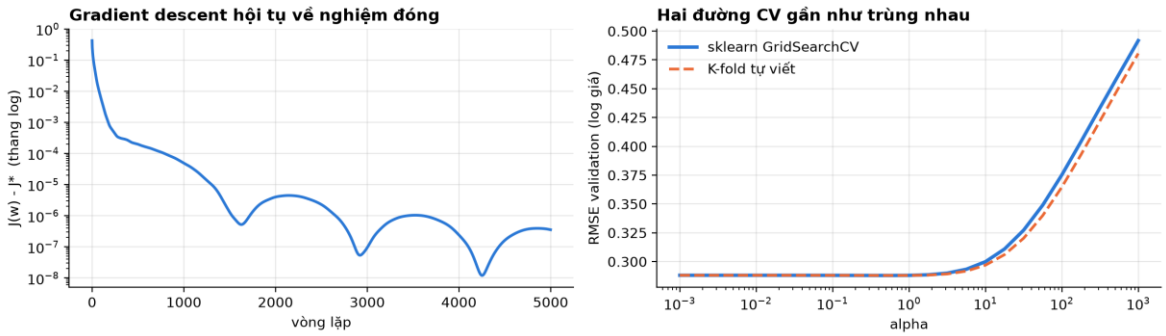
Chọn α bằng **K-fold CV tự viết** (xáo trộn chỉ số rồi chia thành K phần). Lưu ý: `sklearn` tối thiểu hóa $\|y - Aw\|^2 + \alpha\|w\|^2$ (không chia n), còn bản tự viết chia n, nên hai mô hình là một khi α(tự viết) = α(sklearn) / n. K-fold tự viết chọn đúng **α = 0,5623**, cùng CV RMSE 0,2879.

4.3.4. Kiểm chứng

Bảng 4.3. *Kiểm chứng Ridge tự cài đặt với bản thư viện*

Kiểm tra	Kết quả
Nghiệm đóng so với `sklearn` (cùng α): lệch hệ số lớn nhất	$2{,}8 \times 10^{-11}$
Gradient descent (5.000 vòng, 17,1 giây): J − J*	$3{,}6 \times 10^{-7}$
Gradient descent: lệch dự đoán log giá lớn nhất trên test	$2{,}1 \times 10^{-3}$
MAPE test: thư viện / nghiệm đóng / GD	0,1847 / 0,1847 / 0,1847

Hệ số của nghiệm đóng trùng `sklearn` đến sai số làm tròn của máy tính, gradient descent hội tụ về cùng nghiệm. Vậy bản tự cài đặt **đúng**.



Hình 4.3. *Kiểm chứng Ridge: đường CV tự viết trùng thư viện và quá trình hội tụ của gradient descent*

4.4. Mô hình 2: Random Forest

4.4.1. Cơ sở lý thuyết

Thành phần: cây hồi quy CART. Cây chia dữ liệu bằng các câu hỏi có/không, ví dụ "tuổi xe ≤ 5?", "dòng xe là Toyota Vios?". Tại mỗi nút, thuật toán thử các đặc trưng và ngưỡng, chọn phép chia làm **giảm tổng bình phương sai số (SSE)** nhiều nhất. Với nút có n mẫu, tổng nhãn S, chia thành trái (n_L, S_L) và phải (n_R, S_R), mức giảm SSE là:

$$\text{gain} = \frac{S_L^2}{n_L} + \frac{S_R^2}{n_R} - \frac{S^2}{n}$$

Lặp lại đến khi đạt độ sâu tối đa hoặc nút quá ít mẫu; giá trị dự đoán ở lá là trung bình log giá của các xe trong lá. Cây sâu học thuộc dữ liệu train (độ chệch thấp) nhưng **phương sai cao**: đổi vài mẫu là cấu trúc cây thay đổi hẳn.

Random Forest giảm phương sai bằng cách lấy trung bình nhiều cây khác nhau:

1. **Bootstrap:** mỗi cây học trên một mẫu lấy ngẫu nhiên **có hoàn lại** từ train (khoảng 63% mẫu khác nhau).
2. **Chọn ngẫu nhiên đặc trưng:** mỗi lần chia nút chỉ xét một tỉ lệ `max_features` đặc trưng, để các cây ít giống nhau.
3. **Lấy trung bình** dự đoán của tất cả các cây.

$$\hat{y}(x) = \frac{1}{T}\sum_{t=1}^{T} f_t(x)$$

Out-of-bag (OOB): khoảng 37% mẫu không được chọn vào bootstrap của mỗi cây. Dự đoán các mẫu này bằng những cây "chưa thấy" chúng cho ra một ước lượng sai số khách quan mà không cần tập validation riêng.

Bảng 4.4. *Siêu tham số của Random Forest*

Siêu tham số	Tác dụng
`n_estimators`	Số cây; nhiều hơn thì ổn định hơn, đến một mức thì bão hòa
`max_depth`	Độ sâu tối đa của cây
`min_samples_leaf`	Số mẫu tối thiểu ở mỗi lá; lớn hơn thì cây "mượt" hơn
`max_features`	Tỉ lệ đặc trưng xét ở mỗi lần chia; nhỏ hơn thì các cây đa dạng hơn

4.4.2. Bản thư viện

Dùng `RandomForestRegressor`, tinh chỉnh bằng **RandomizedSearchCV** (15 cấu hình ngẫu nhiên) + **3-fold CV** trên train, mất 156 giây. Cấu hình tốt nhất: **300 cây, `max_depth` = 30, `min_samples_leaf` = 1, `max_features` = 0,5**; CV RMSE (log) = 0,2741; OOB R^2 (log) = 0,8956; huấn luyện 7,9 giây.

Bảng 4.5. *Năm cấu hình Random Forest tốt nhất (thư viện)*

n_estimators	min_samples_leaf	max_features	max_depth	CV RMSE (log)
300	**1**	**0,50**	**30**	**0,2741**
300	1	0,33	30	0,2747
300	2	0,50	30	0,2819
300	2	0,80	30	0,2827
300	2	0,33	30	0,2839

Các cấu hình tốt đều có cây sâu và lá nhỏ: dữ liệu có nhiều tổ hợp hãng-dòng-đời xe riêng biệt nên mô hình cần đủ độ phức tạp để học từng nhóm.

4.4.3. Bản tự cài đặt bằng NumPy

Bảng 4.6. *Các lớp của Random Forest tự cài đặt*

Lớp	Nhiệm vụ
`Binner`	Rời rạc hóa mỗi đặc trưng thành tối đa 32 bin theo phân vị (fit trên train)
`RegressionTreeScratch`	Cây hồi quy CART: tìm phép chia tốt nhất theo công thức gain
`RandomForestScratch`	Bootstrap + chọn ngẫu nhiên đặc trưng + lấy trung bình; tính OOB và độ quan trọng đặc trưng

Tăng tốc bằng histogram. Cách cơ bản phải sắp xếp từng cột ở mỗi nút để thử mọi ngưỡng, rất chậm khi viết bằng Python. Thay vào đó, mỗi đặc trưng được chia sẵn thành tối đa 32 bin **một lần** (kỹ thuật của LightGBM / HistGradientBoosting). Tại mỗi

nút chỉ cần đếm số mẫu và tổng nhãn trong từng bin (`np.bincount`), rồi cộng dồn (`np.cumsum`) là có gain của **mọi ngưỡng cùng lúc**. Cột one-hot chỉ có 2 giá trị nên không mất thông tin; cột số mất rất ít độ chi tiết.

Độ quan trọng đặc trưng = tổng gain mà đặc trưng mang lại ở mọi nút của mọi cây, chuẩn hóa về tổng bằng 1 (cùng định nghĩa với `sklearn`).

4.4.4. Kiểm chứng thành phần: một cây

Trước khi dựng rừng, so sánh **một cây** tự viết (xét mọi đặc trưng ở mỗi nút) với `DecisionTreeRegressor` khi dùng **cùng tham số**:

Bảng 4.7. *Kiểm chứng cây hồi quy tự cài đặt*

Mô hình	Số nút	R^2 (log) train	R^2 (log) test	MAPE test	Thời gian
Một cây – thư viện	3.481	0,929	0,842	19,6%	0,3 s
Một cây – tự cài đặt	3.429	0,927	0,851	18,6%	1,6 s

Hai cây có số nút và R^2 gần như nhau, nên **lớp cây tự viết đúng**. Khoảng cách lớn giữa R^2 train (0,93) và R^2 test (0,84-0,85) chính là **phương sai cao** mà Random Forest khắc phục. Bảng này dùng tập test chỉ để kiểm tra tính đúng của code, không dùng để chọn tham số.

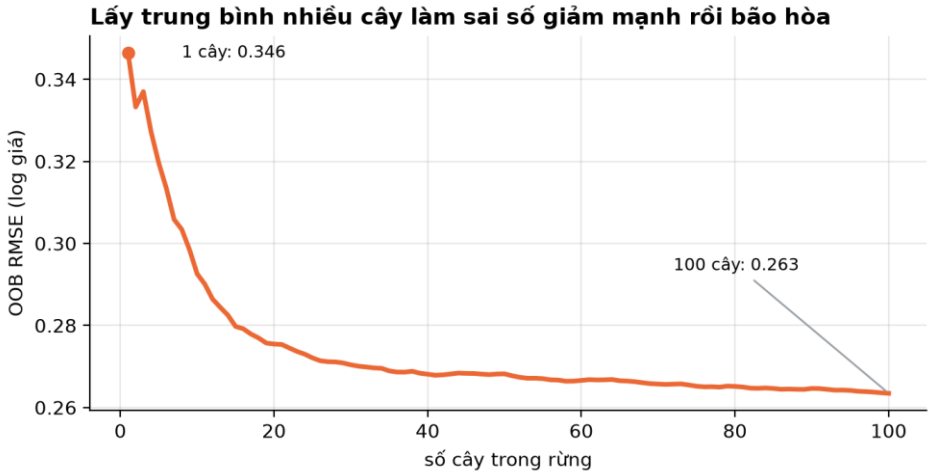
4.4.5. Tinh chỉnh bằng OOB và chọn số cây

Bản tự viết chậm hơn thư viện nhiều, nên thay vì CV (huấn luyện 3-5 lần mỗi cấu hình), dùng **OOB RMSE**: mỗi cấu hình chỉ huấn luyện một lần với 30 cây. Thử 6 cấu hình trong cùng không gian tham số, mất 151 giây.

Bảng 4.8. *Tinh chỉnh Random Forest tự cài đặt bằng OOB (30 cây)*

max_depth	min_samples_leaf	max_features	OOB RMSE (log)	OOB R^2	Thời gian (s)
30	**2**	**0,50**	**0,2704**	**0,886**	**36**
30	2	0,33	0,2741	0,883	28
30	3	0,33	0,2779	0,879	21
20	1	0,33	0,2938	0,865	24
20	2	0,33	0,2946	0,864	19
20	3	0,50	0,2972	0,862	23

Với cấu hình tốt nhất, huấn luyện **100 cây** mất 152 giây, OOB R^2 (log) = **0,8916** (bản thư viện 0,8956). Hình 4.4 cho thấy trực quan vì sao "rừng" tốt hơn "cây": sai số giảm rất nhanh trong vài chục cây đầu rồi bão hòa.



Hình 4.4. *OOB RMSE của rừng tự cài đặt theo số cây*

4.5. Kỹ thuật xử lý mất cân bằng

Ba kỹ thuật được thử trên chính mô hình cuối (Random Forest), so với huấn luyện bình thường:

Bảng 4.9. *Các kỹ thuật xử lý mất cân bằng được thử nghiệm*

Kỹ thuật	Cách làm (chỉ trong phần train của mỗi fold)
Không cân bằng	Huấn luyện bình thường
Trọng số mẫu	Mẫu ở phân khúc hiếm có trọng số lớn hơn: 1/√(tần suất phân khúc), chuẩn hóa về trung bình 1
Random Oversampling	Nhân bản ngẫu nhiên mẫu của phân khúc hiếm đến tối thiểu 25% quy mô phân khúc lớn nhất
Random Undersampling	Bỏ bớt mẫu ở phân khúc đông, mỗi phân khúc giữ tối đa 8 lần số mẫu của phân khúc hiếm nhất

Cách đánh giá: 5-fold CV phân tầng theo phân khúc giá **chỉ trên tập train**. Việc cân bằng chỉ áp dụng cho phần train của từng fold; phần validation **giữ nguyên phân phối thật**. Cân bằng trước khi chia sẽ làm bản sao của cùng một xe nằm ở cả hai phía và cho điểm cao giả tạo. Thí nghiệm cũng so sánh học trên giá gốc với học trên log giá. Tập test không được dùng ở bước này.

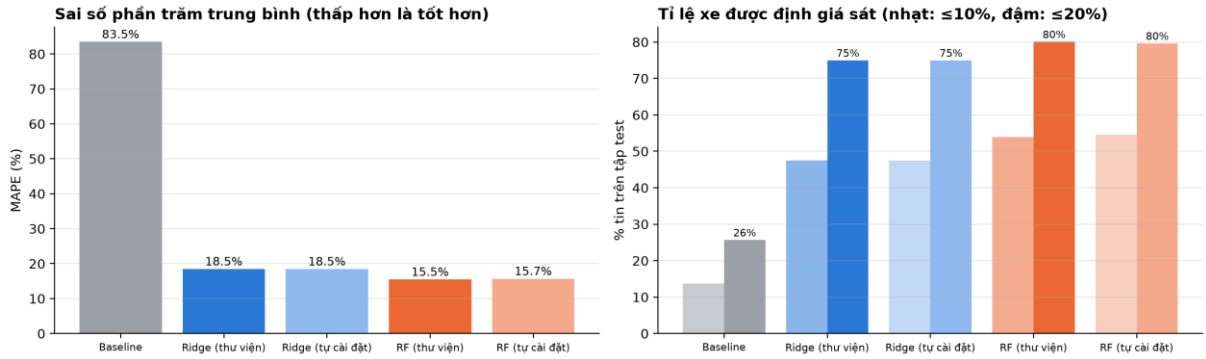
CHƯƠNG 5. KẾT QUẢ VÀ ĐÁNH GIÁ

5.1. Kết quả tổng thể trên tập test

Đây là lần **duy nhất** tập test được dùng. Mọi mô hình học trên cùng tập train (11.149 tin) và được đánh giá trên cùng 2.788 tin test.

Bảng 5.1. *Kết quả trên tập test*

Mô hình	MAPE	MedAPE	MAE (tr)	RMSE (tr)	R^2 (log)	±10%	±20%	Fit (s)
Baseline (trung vị)	83,5%	40,0%	337	724	0,00	14%	26%	0,001
Ridge – thư viện	18,5%	10,7%	120	411	0,873	48%	75%	0,04
Ridge – tự cài đặt (nghiệm đóng)	18,5%	10,7%	120	411	0,873	48%	75%	0,02
Ridge – tự cài đặt (GD)	18,5%	10,7%	120	411	0,873	48%	75%	17,1
Random Forest – thư viện	**15,5%**	**9,0%**	**102**	**372**	**0,891**	**54%**	**80%**	**7,9**
Random Forest – tự cài đặt	15,7%	9,0%	106	388	0,887	55%	80%	151,7



Hình 5.1. *So sánh MAPE và tỉ lệ xe được định giá sát giữa các mô hình*

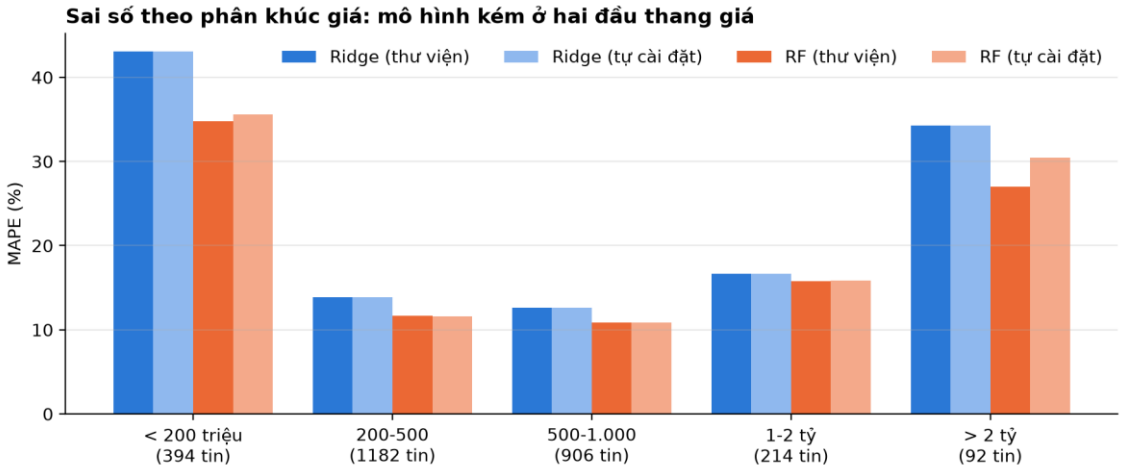
- **Cả hai mô hình vượt xa mốc so sánh:** baseline sai trung bình 83,5%; Ridge giảm còn 18,5%, Random Forest còn 15,5%.
- Với Random Forest, **4 trên 5 xe** được định giá lệch không quá 20%, hơn một nửa lệch không quá 10%, sai số điển hình (MedAPE) khoảng 9%.
- Bản tự cài đặt cho kết quả gần như trùng bản thư viện; khác biệt lớn nhất là thời gian huấn luyện.

5.2. Sai số theo phân khúc giá

Bảng 5.2. *MAPE theo phân khúc giá trên tập test (TV: thư viện, TCĐ: tự cài đặt)*

Phân khúc	% dữ liệu	Baseline	Ridge	Random Forest (TV)	Random Forest (TCĐ)
< 200 triệu	14%	349,8%	43,0%	34,7%	35,6%

Phân khúc	% dữ liệu	Baseline	Ridge	Random Forest (TV)	Random Forest (TCĐ)
200-500 triệu	42%	38,5%	13,9%	11,7%	11,6%
500-1.000 triệu	33%	30,7%	12,6%	10,9%	10,8%
1-2 tỷ	8%	64,1%	16,7%	15,7%	15,8%
> 2 tỷ	3%	85,4%	34,3%	27,0%	30,4%

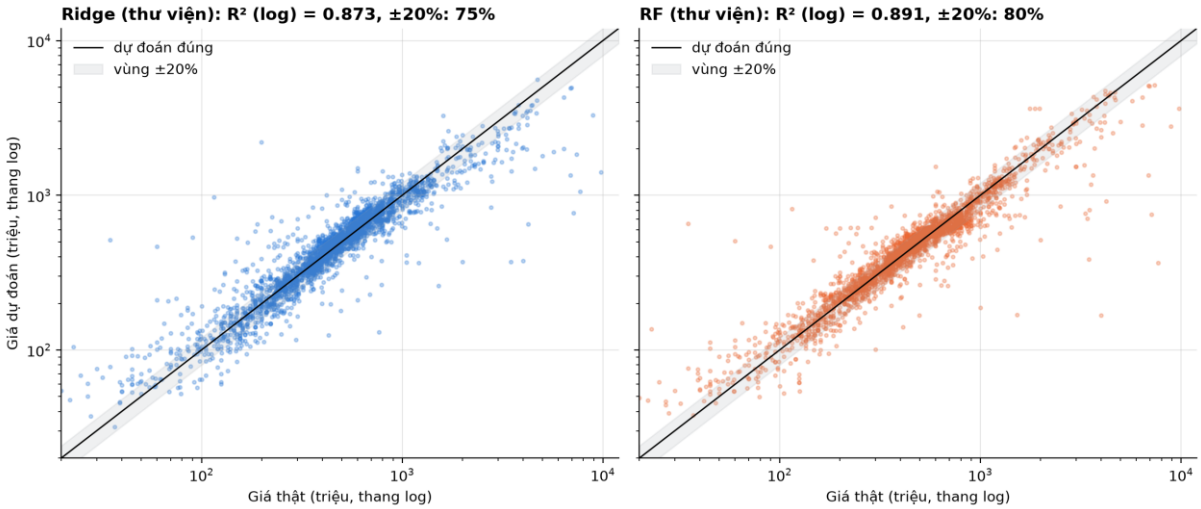


Hình 5.2. *MAPE theo phân khúc giá*

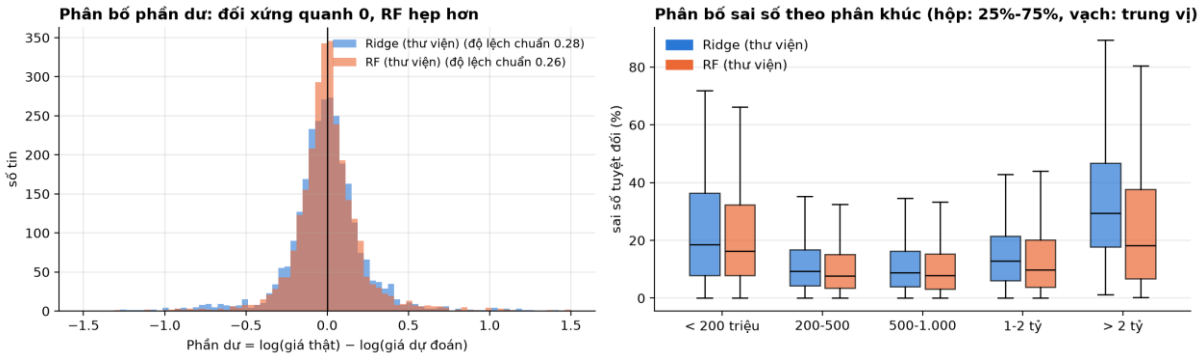
- **Phân khúc giữa (200 triệu - 1 tỷ, khoảng 75% thị trường):** sai số khoảng 11%, đủ tốt để tham khảo giá.
- **Xe dưới 200 triệu:** sai số phần trăm cao vì giá thấp: chênh vài chục triệu (tình trạng thực tế, thương lượng) đã là phần trăm lớn; xe đời cũ rất khác nhau về tình trạng mà dữ liệu không có; có thể vẫn còn tin ghi tiền trả trước chưa bị bắt.
- **Xe trên 2 tỷ:** chỉ 3% dữ liệu và thiếu thông tin quyết định giá xe sang (phiên bản, trang bị).
- Random Forest tốt hơn Ridge ở **mọi phân khúc**.

5.3. Dự đoán so với giá thật và phần dư

Trên Hình 5.3, mỗi điểm là một xe trong tập test; trục dùng thang log nên khoảng cách tới đường chéo chính là sai số phần trăm. Phần dư của cả hai mô hình đối xứng quanh 0 (không lệch hệ thống), Random Forest có phân bố hẹp hơn (độ lệch chuẩn 0,26 so với 0,28). Biểu đồ hộp cho thấy sai số tăng ở hai đầu thang giá, rõ nhất ở xe trên 2 tỷ.



Hình 5.3. *Giá dự đoán so với giá thật trên tập test (thang log)*



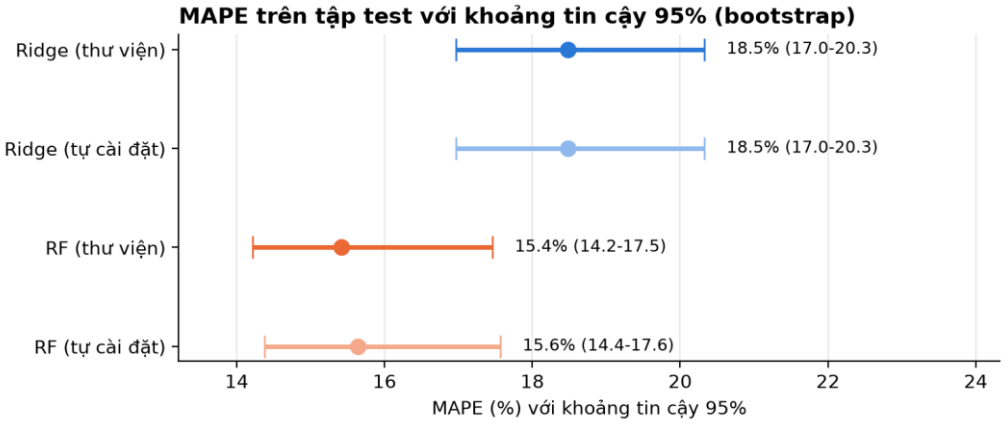
Hình 5.4. *Phân bố phần dư và sai số tuyệt đối theo phân khúc*

5.4. Kiểm định ý nghĩa thống kê bằng bootstrap

Tập test hữu hạn nên MAPE chỉ là một ước lượng. **Bootstrap** lấy mẫu lại tập test có hoàn lại 1.000 lần, mỗi lần tính lại MAPE, để có khoảng tin cậy 95%. Để so sánh hai mô hình, dùng **bootstrap ghép cặp**: trong mỗi lần lấy mẫu, tính **hiệu MAPE của hai mô hình trên cùng các xe**. Nếu khoảng tin cậy 95% của hiệu không chứa 0 thì chênh lệch có ý nghĩa thống kê.

Bảng 5.3. *Hiệu MAPE giữa các mô hình (điểm %), bootstrap ghép cặp*

So sánh	2,5%	Trung vị	97,5%	Có ý nghĩa?
Ridge − Random Forest (thư viện)	2,08	2,97	3,97	**Có**
RF tự cài đặt − RF thư viện	−0,04	0,19	0,46	Không (chứa 0)
Ridge tự cài đặt − Ridge thư viện	0,00	0,00	0,00	Không (cùng nghiệm)

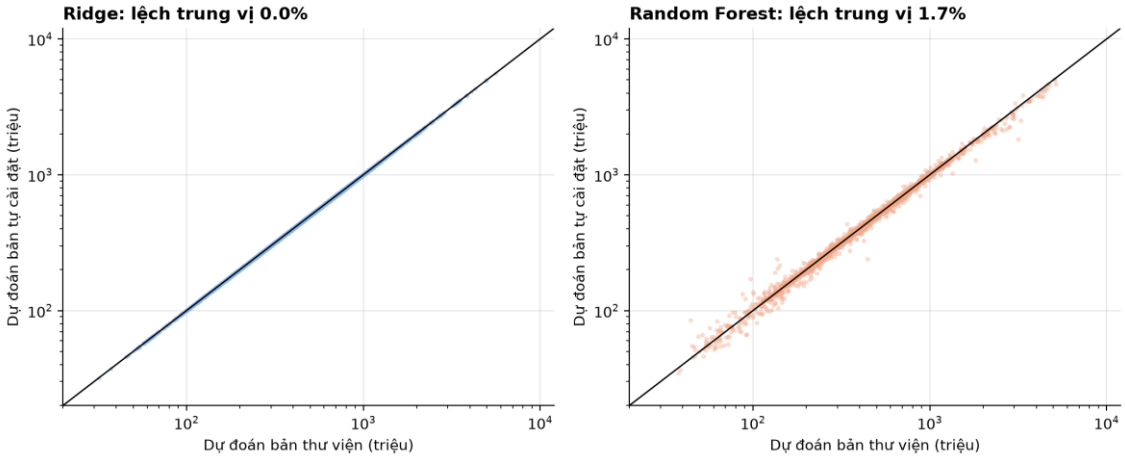


Hình 5.5. *Khoảng tin cậy 95% của MAPE và của hiệu MAPE*

Random Forest tốt hơn Ridge có ý nghĩa thống kê (khoảng 3 điểm %, KTC 95%: 2,1-4,0). **Bản tự cài đặt và bản thư viện không khác nhau có ý nghĩa.**

5.5. Bản tự cài đặt so với bản thư viện

Nếu hai bản cài đặt cùng một thuật toán, dự đoán của chúng phải gần như trùng nhau trên **từng xe**, không chỉ ở con số trung bình. Hình 5.6 cho thấy dự đoán của Ridge trùng tuyệt đối; với Random Forest, các điểm bám sát đường chéo với độ lệch trung vị 1,7%. Chênh lệch nhỏ của Random Forest đến từ việc chia đặc trưng thành 32 bin, rừng tự viết dùng 100 cây thay vì 300, và chuỗi số ngẫu nhiên khác nhau.



Hình 5.6. *Dự đoán của bản tự cài đặt so với bản thư viện trên từng xe trong tập test*

Bảng 5.4. *Tóm tắt so sánh bản tự cài đặt và bản thư viện*

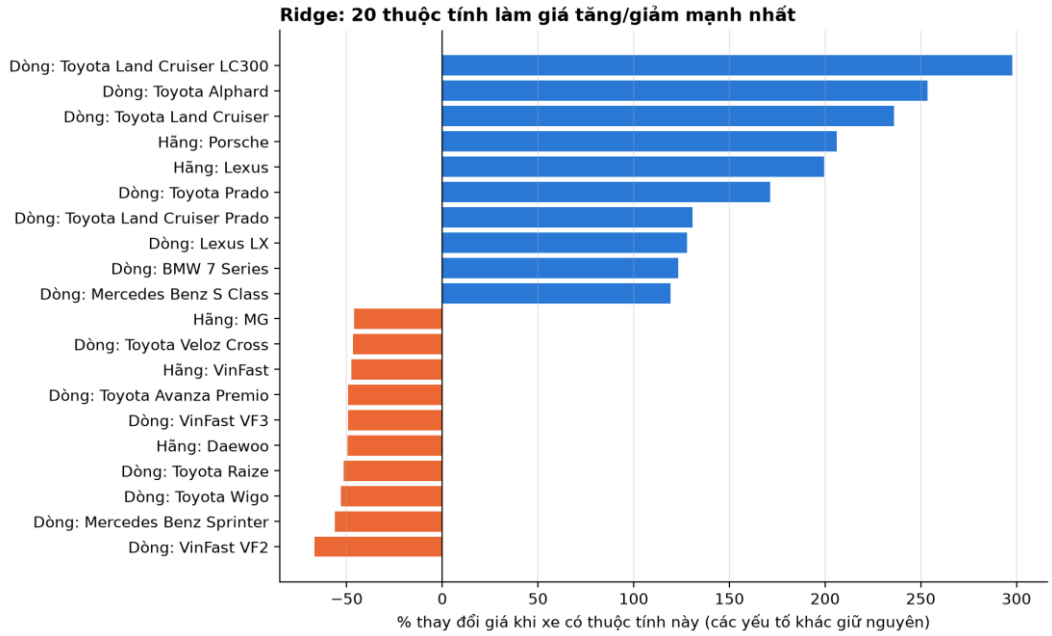
Tiêu chí	Ridge	Random Forest
MAPE test (thư viện / tự cài đặt)	18,5% / 18,5%	15,5% / 15,7%
Kiểm chứng	Hệ số lệch ~10^{-11}; GD hội tụ cùng nghiệm	OOB R^2 0,892 so với 0,896; cùng thứ hạng đặc trưng
Thời gian huấn luyện	0,02 s (nghiệm đóng), 17 s (GD) so với 0,04 s	152 s (100 cây) so với 7,9 s (300 cây)

Chi phí tính toán là khác biệt lớn nhất: thư viện viết bằng C/Cython và chạy song song nhiều lõi. Đây là lý do thực tế để dùng thư viện khi triển khai, còn bản tự cài đặt có giá trị để hiểu thuật toán.

5.6. Diễn giải mô hình

5.6.1. Hệ số của Ridge

Mỗi hệ số của cột one-hot cho biết thuộc tính đó làm giá cao hơn hay thấp hơn bao nhiêu % so với mức chung, khi giữ nguyên các đặc trưng khác. Vì mô hình có **cả** cột hãng **và** cột dòng xe, hệ số của một dòng xe là phần **chênh thêm so với mức chung của hãng đó**. Ví dụ hệ số âm của `Hãng: VinFast` nghĩa là xe VinFast có giá thấp hơn mức chung của các xe cùng tuổi, cùng số km (nhiều khả năng do xe điện mất giá nhanh trên thị trường xe cũ).



Hình 5.7. *Các hệ số Ridge ảnh hưởng mạnh nhất*

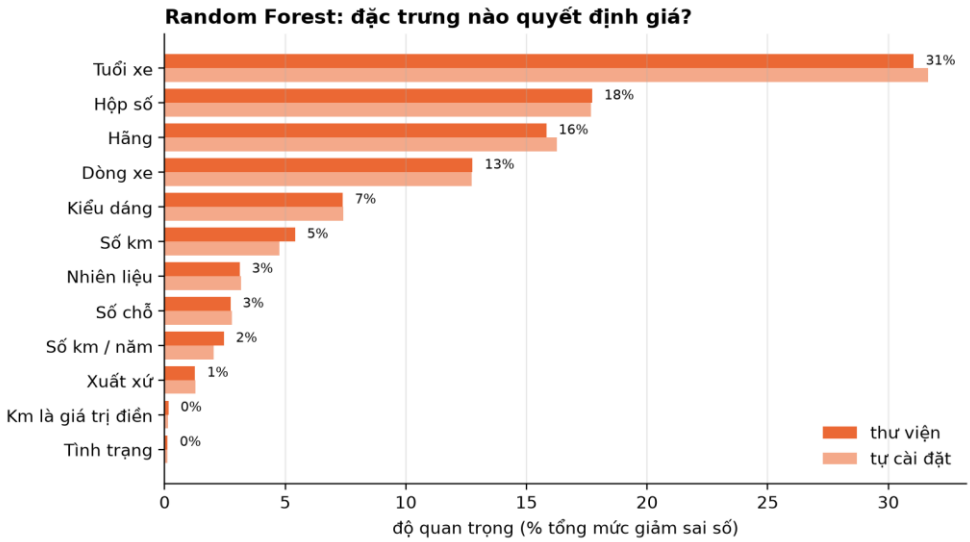
Hệ số của các đặc trưng số (đã chuẩn hóa): `age` −0,987, `age²` +0,295, `log_km` +0,705, `log_km_per_year` −0,621, `carseats` +0,031, `mileage_missing` −0,008. Các hệ số này khó đọc riêng lẻ vì `age` và `age²` (hay km và km/năm) cùng mô tả một hiện tượng; cách đọc đúng là xem đường khấu hao (mục 5.6.3).

5.6.2. Độ quan trọng đặc trưng của Random Forest

Một đặc trưng phân loại bị one-hot thành nhiều cột, nên độ quan trọng của mọi cột thuộc cùng một đặc trưng gốc được cộng lại:

Bảng 5.5. *Độ quan trọng đặc trưng của Random Forest (gộp theo đặc trưng gốc)*

Đặc trưng	Thư viện (%)	Tự cài đặt (%)
Tuổi xe	31,0	31,7
Hộp số	17,7	17,7
Hãng	15,8	16,3
Dòng xe	12,8	12,7
Kiểu dáng	7,4	7,4
Số km	5,4	4,7
Nhiên liệu	3,1	3,2
Số chỗ	2,7	2,8
Số km / năm	2,5	2,0
Xuất xứ	1,3	1,3
Km là giá trị điền	0,2	0,1
Tình trạng	0,1	0,1



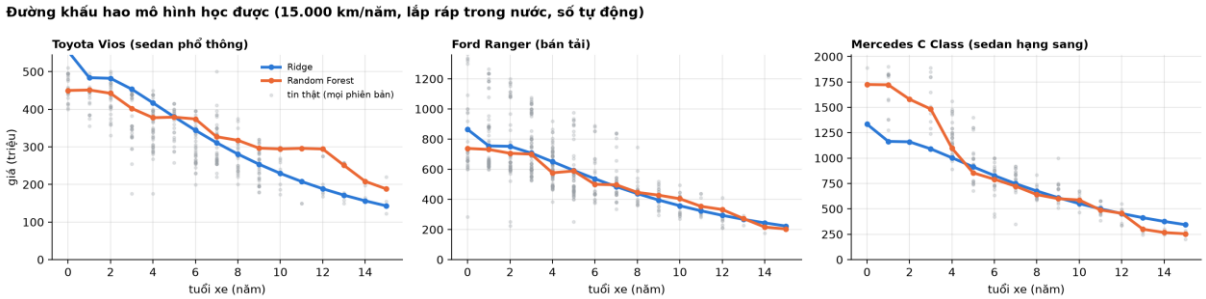
Hình 5.8. *Độ quan trọng đặc trưng: bản thư viện và bản tự cài đặt*

- **Tuổi xe** quan trọng nhất (khoảng 1/3 tổng): khấu hao là yếu tố lớn nhất quyết định giá xe cũ.
- **Hãng và dòng xe** cộng lại gần ngang tuổi xe: hai xe cùng tuổi nhưng khác dòng có thể chênh nhau hàng chục lần.
- **Hộp số** đứng thứ hai, cao hơn trực giác. Nhiều khả năng đây là **đại diện gián tiếp**: xe số sàn thường là xe đời cũ, xe tải nhẹ, phân khúc thấp, nên câu hỏi "số sàn hay tự động?" tách nhanh nhóm xe rẻ khỏi nhóm xe đắt. Độ quan trọng đo mức đóng góp vào việc chia dữ liệu, không phải tác động nhân quả.
- **Tình trạng** gần như bằng 0 vì thông tin đã nằm trong `tuổi = 0` và `số km = 0`.

- Hai bản cài đặt cho **cùng thứ hạng** và gần như cùng tỉ lệ, thêm một bằng chứng rừng tự viết hoạt động đúng.

5.6.3. Đường khấu hao

Lấy ba mẫu xe phổ biến ở ba phân khúc, cố định mọi thuộc tính, chỉ thay đổi tuổi xe (giả định chạy 15.000 km/năm; tuổi 0 là xe mới, 0 km), rồi xem hai mô hình dự đoán giá:



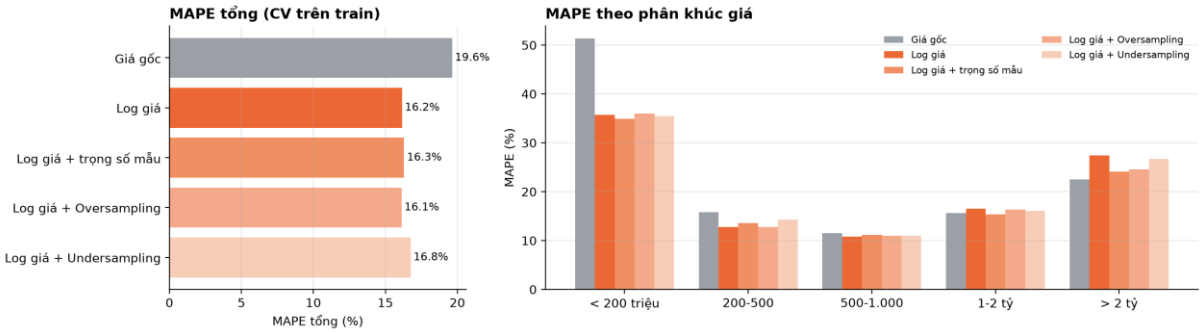
Hình 5.9. *Giá dự đoán theo tuổi xe của ba mẫu xe ở ba phân khúc*

- **Ridge** cho đường khấu hao trơn và **cùng một hình dạng** cho mọi mẫu xe (chỉ dịch lên xuống), vì mô hình tuyến tính trên log giá giả định mọi xe mất giá cùng một tỉ lệ % mỗi năm.
- **Random Forest** cho đường bậc thang với **hình dạng khác nhau giữa các dòng xe**: xe sang (Mercedes C Class) mất giá rất nhanh trong 3-4 năm đầu, khác hẳn xe phổ thông. Đây chính là **tương tác** giữa dòng xe và tuổi xe mà Ridge không biểu diễn được, và là lý do Random Forest chính xác hơn.
- Các điểm xám là tin thật của cùng dòng xe; độ phân tán dọc lớn cho thấy phần giá mà dữ liệu không giải thích được (phiên bản, trang bị, tình trạng thực tế).

5.7. Thí nghiệm log giá và xử lý mất cân bằng

Bảng 5.6. *Thí nghiệm trên Random Forest (5-fold CV trên tập train)*

Phương án	Cỡ train/fold	MAPE	MedAPE	< 200 tr	200-500	500-1.000	1-2 tỷ	> 2 tỷ
Giá gốc	8.919	19,6%	9,7%	51,3%	15,8%	11,5%	15,6%	22,5%
Log giá (chọn)	**8.919**	**16,2%**	**9,1%**	**35,7%**	**12,8%**	**10,8%**	**16,5%**	**27,4%**
Log + trọng số mẫu	8.919	16,3%	8,8%	34,9%	13,6%	11,2%	15,3%	24,1%
Log + Oversampling	9.830	16,1%	8,7%	36,0%	12,8%	11,0%	16,3%	24,5%
Log + Undersampling	6.925	16,8%	9,5%	35,5%	14,3%	11,0%	16,0%	26,7%



Hình 5.10. *MAPE theo phân khúc của các phương án (CV trên train)*

Log giá tốt hơn rõ rệt. MAPE tổng giảm từ 19,6% xuống 16,2%; phân khúc dưới 200 triệu cải thiện mạnh nhất (51% → 36%). Học trên giá gốc thì sai số bình phương bị xe đắt chi phối, nên mô hình bỏ qua sai số tương đối ở xe rẻ. Đổi lại, giá gốc tốt hơn ở xe trên 2 tỷ (22,5% so với 27,4%), cũng vì lý do đó.

Các kỹ thuật cân bằng không thay đổi MAPE tổng, nhưng giúp phân khúc hiếm:

- MAPE tổng của ba kỹ thuật nằm trong khoảng 16,1-16,8%, so với 16,2% khi không cân bằng, trong mức dao động của CV.
- **Oversampling** và **trọng số mẫu** giảm sai số ở xe trên 2 tỷ khoảng 3 điểm % (27,4% → 24-25%). Trọng số mẫu phải đổi lại bằng sai số cao hơn ở phân khúc 200-500 triệu (12,8% → 13,6%); oversampling gần như không làm phân khúc nào kém đi.
- **Undersampling** bỏ khoảng 22% dữ liệu train, MAPE tổng tăng lên 16,8% và gần như không giúp xe sang: bỏ dữ liệu thật thì mất thông tin.
- Dù đã cân bằng, sai số ở xe trên 2 tỷ vẫn cao hơn gấp đôi phân khúc giữa: nguyên nhân chủ yếu là **thiếu thông tin** (phiên bản, trang bị), không chỉ do ít mẫu. Nhân bản dữ liệu không tạo thêm thông tin mới.

Quyết định: mô hình cuối **học trên log giá, không dùng kỹ thuật cân bằng**, vì thước đo chính không đổi và mô hình đơn giản hơn. Nếu ứng dụng ưu tiên định giá xe sang, nên dùng **Random Oversampling** (chỉ trên tập train).

5.8. Ảnh hưởng của tin đăng lại

Bước tiền xử lý đã xóa các tin trùng hoàn toàn, nhưng cùng một chiếc xe có thể được **đăng lại với giá chênh nhẹ**. Khi chia ngẫu nhiên, một bản rơi vào train và bản kia vào test; Random Forest với lá nhỏ chỉ cần "nhớ" giá là đoán đúng. Kiểm tra: tin test nào có một tin train **giống hệt mọi đặc trưng, kể cả số km**?

Bảng 5.7. *Sai số trên nhóm tin có và không có bản sao trong tập train*

Nhóm tin test	Số tin	% test	MAPE Ridge	MAPE RF
Có bản sao trong train	391	14,0%	14,0%	10,9%
trong đó xe đã dùng	247	8,9%	14,0%	10,5%
Không có bản sao	2.397	86,0%	19,2%	16,2%
Toàn bộ test	2.788	100%	18,5%	15,5%

MAPE trên nhóm **không có bản sao** (RF 16,2%) là ước lượng thận trọng hơn cho một chiếc xe hoàn toàn mới với mô hình; con số 15,5% hơi lạc quan. Cách chặt chẽ hơn là chia train/test theo nhóm (`StratifiedGroupKFold`) để mọi bản đăng của một xe nằm cùng một phía.

5.9. Lưu và sử dụng mô hình

Các mô hình đã huấn luyện được lưu trong thư mục `model/` để dùng lại mà không phải huấn luyện lại:

Bảng 5.8. *Các file mô hình đã lưu*

File	Nội dung
`preprocessor_ridge.joblib`, `preprocessor_rf.joblib`	Bộ mã hóa one-hot + chuẩn hóa của từng mô hình (fit trên train)
`ridge_sklearn.joblib`, `random_forest_sklearn.joblib`	Mô hình thư viện
`ridge_scratch.npz`, `random_forest_scratch.npz`	Trọng số Ridge tự cài đặt; các cây và ngưỡng bin của rừng tự cài đặt (mảng NumPy)
`metadata.json`	Danh sách đặc trưng, bảng gộp nhãn, trung vị số km theo năm, tham số tốt nhất, metric
`price_model.py`	Hàm `load_model` / `predict_price`: nhận thông tin thô của xe, tự dựng đặc trưng như lúc huấn luyện

Khi nạp lại bằng `price_model.py` và truyền thông tin thô của các xe test, dự đoán trùng với notebook (lệch lớn nhất $1{,}6 \times 10^{-11}$ triệu). Ví dụ dự đoán:

Bảng 5.9. *Ví dụ dự đoán giá (triệu đồng)*

Xe	Ridge (TV)	Ridge (TCĐ)	RF (TV)	RF (TCĐ)
Toyota Vios 2018, 80.000 km	281	281	303	303
Ford Ranger 2021, không rõ số km	596	596	515	536

CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1. Kết quả đạt được

- Xây dựng hoàn chỉnh quy trình học máy từ đầu đến cuối: thu thập **14.835 tin** từ Chợ Tốt, tiền xử lý còn **13.937 tin** sạch, huấn luyện, đánh giá, diễn giải và đóng gói mô hình để sử dụng.
- Tiền xử lý dựa trên hiểu biết về cách dữ liệu được tạo ra: phát hiện giá giả, giá trả trước/đặt cọc, xe "mới" ghi sai, số km nhập sai đơn vị; điền thiếu theo cơ chế thiếu; gộp nhãn hiếm theo cấp với ngưỡng chọn bằng CV.
- **Random Forest** là mô hình tốt nhất: MAPE **15,5%**, MedAPE 9,0%, R^2 (log) 0,891, **80% xe** được định giá lệch không quá 20%. **Ridge** đạt MAPE 18,5%. Chênh lệch khoảng 3 điểm % **có ý nghĩa thống kê** (KTC 95%: 2,1-4,0), và là phần đóng góp của quan hệ phi tuyến và tương tác giữa các đặc trưng.
- Cả hai mô hình được **tự cài đặt bằng NumPy** và kiểm chứng với `scikit-learn`: Ridge trùng đến sai số làm tròn; Random Forest (histogram + CART + bootstrap + OOB) đạt MAPE 15,7%, khác bản thư viện không có ý nghĩa thống kê.
- Các quyết định về dữ liệu đều kiểm chứng bằng CV trên tập train: học trên log giá giảm MAPE từ 19,6% xuống 16,2%; trọng số mẫu và oversampling giữ nguyên MAPE tổng nhưng giảm khoảng 3 điểm % sai số ở xe trên 2 tỷ; undersampling làm kết quả kém đi.
- Yếu tố quyết định giá: tuổi xe (~31%), hộp số (~18%), hãng (~16%), dòng xe (~13%), kiểu dáng (~7%).

6.2. Hạn chế

- **Giá rao bán, không phải giá giao dịch:** luôn có phần thương lượng mà mô hình không biết.
- **Thiếu đặc trưng quan trọng:** phiên bản/trang bị, tình trạng thân vỏ và nội thất, số đời chủ. Đây là nguyên nhân chính của sai số cao ở xe trên 2 tỷ (27%) và xe dưới 200 triệu (35%).
- **Tin đăng lại:** khoảng 14% tin test có bản sao trong train; trên phần còn lại MAPE của Random Forest là 16,2%.
- **Rò rỉ nhẹ trong tiền xử lý:** việc điền thiếu dùng thống kê trên toàn bộ dữ liệu trước khi chia (chỉ dựa trên đặc trưng, không dùng giá); bộ lọc giá trả trước dùng giá trên toàn bộ dữ liệu (là bước sửa nhãn). Rủi ro thấp nhưng cần nêu rõ.
- Giá dự đoán là `exp()` của dự đoán log, tức ước lượng **trung vị** của giá, hơi thấp hơn giá trung bình; phù hợp với MAPE nên không hiệu chỉnh.

- Dữ liệu là ảnh chụp tại một thời điểm (10/2026); mô hình cần được huấn luyện lại định kỳ khi thị trường thay đổi.

6.3. Hướng phát triển

- Crawl thêm phần mô tả tin đăng để trích phiên bản, trang bị, và lọc giá trả trước theo từ khóa ("trả trước", "góp", "cọc").
- Chia train/test theo nhóm (`StratifiedGroupKFold`) để loại ảnh hưởng của tin đăng lại.
- Thử **Gradient Boosting** (học tuần tự các cây sửa lỗi của nhau), có thể dùng lại lớp cây đã tự cài đặt.
- Thử hàm mất mát theo phần trăm để cải thiện hai đầu thang giá.
- Xây dựng giao diện web nhỏ cho phép người dùng nhập thông tin xe và nhận giá tham khảo, dựa trên `price_model.py`.

TÀI LIỆU THAM KHẢO

[1] Hoerl, A. E., & Kennard, R. W. (1970). Ridge regression: Biased estimation for nonorthogonal problems. *Technometrics*, 12(1), 55–67.

[2] Breiman, L., Friedman, J., Olshen, R., & Stone, C. (1984). *Classification and Regression Trees*. Wadsworth.

[3] Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32.

[4] Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning* (2nd ed.). Springer.

[5] Ke, G., Meng, Q., Finley, T., et al. (2017). LightGBM: A highly efficient gradient boosting decision tree. *Advances in Neural Information Processing Systems 30*.

[6] Nesterov, Y. (1983). A method for solving the convex programming problem with convergence rate O(1/k²). *Soviet Mathematics Doklady*, 27, 372–376.

[7] Efron, B., & Tibshirani, R. J. (1993). *An Introduction to the Bootstrap*. Chapman & Hall.

[8] Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

[9] Chợ Tốt Xe. Chuyên mục mua bán ô tô. https://xe.chotot.com/mua-ban-oto (truy cập 25/10/2026).

PHỤ LỤC

A. Cấu trúc mã nguồn

Bảng PL.1. *Cấu trúc thư mục dự án*

Đường dẫn	Nội dung
`src/crawl_data.ipynb`	Thu thập dữ liệu từ Chợ Tốt
`src/visualize_raw_data.ipynb`	Biểu đồ khám phá dữ liệu thô
`src/preprocess_data.ipynb`	Tiền xử lý, tạo `clean_data_oto.csv`
`src/model.ipynb`	Ridge và Random Forest (thư viện + tự cài đặt), đánh giá, thí nghiệm, xuất mô hình
`src/figures/`	Các biểu đồ dùng trong báo cáo
`model/price_model.py`	Nạp mô hình và dự đoán giá từ thông tin thô
`slides/`	Slide trình bày

B. Cách chạy

Chạy các notebook với thư mục làm việc là `src` theo thứ tự: `crawl_data` (không bắt buộc, đã có dữ liệu), `preprocess_data` (khoảng 2 phút), `model` (khoảng 12 phút, phần lớn là rừng tự cài đặt). Dùng mô hình đã lưu:

```
from price_model import load_model, predict_price
import pandas as pd
xe = pd.DataFrame([{"carbrand": "Toyota", "carmodel": "Vios", "carorigin": "Việt
Nam",
    "condition_ad": "Đã sử dụng", "gearbox": "Tự động", "fuel": "Xăng",
    "cartype": "Sedan", "carseats": 5, "mfdate": 2018, "mileage_km": 80000}])
print(predict_price(load_model("random_forest_sklearn"), xe))   # ~303 triệu
```

C. Phân công công việc

Bảng PL.2. *Phân công công việc trong nhóm*

Thành viên	Công việc
<Họ tên SV 1>	<Nội dung công việc>
<Họ tên SV 2>	<Nội dung công việc>